

LEARNING TO THEORIZE IN MENTAL TERMS

MSc Thesis (*Afstudeerscriptie*)

written by

Gidon Kaminer

under the supervision of **Dr Fausto Carcassi**, and submitted to the
Examinations Board in partial fulfilment of the requirements for the degree of

MSc in Logic

at the *Universiteit van Amsterdam*.

Date of the public defence: **Members of the Thesis Committee:**

August 28th, 2026

Dr Fausto Carcassi (supervisor)

Dr Malvin Gatteringer (chair)

Dr Martha Lewis

Dr Giorgio Sbardolini



INSTITUTE FOR LOGIC, LANGUAGE AND COMPUTATION

Abstract

Contents

1	Theory of Mind and its Acquisition	4
1.1	The nature and scope of Theory of Mind	4
1.2	Experimental Evidence: Developmental Milestones	5
1.3	What a good account of ToM acquisition ought to explain	5
1.4	Theoretical perspectives on ToM acquisition	6
1.4.1	Nativism and core knowledge	7
1.4.2	Empiricism, association, and cultural learning	7
1.4.3	Simulationism	8
1.4.4	Rational constructivism and the “theory theory”	9
2	Learning as Bayesian Program Synthesis	10
2.1	The computational-level problem	10
2.2	Bayesian inference: the logic of learning	10
2.3	Programs as the hypothesis space	11
2.4	Program induction and the search problem	13
2.5	Learning the language itself	13
2.6	Our claim	14
3	Bayesian Theory of Mind	16
3.1	Agent as planner	16
3.2	Inverse planning	16
3.3	Belief as an input	17
3.4	What is stipulated by the BToM modeler	18
4	The Model	20
4.1	The learning problem	20
4.2	The task corpus	20
4.3	Program space: type-directed enumeration	25
4.4	The primitive library	25
4.5	Composing belief attribution	27
4.6	Inference	29
4.6.1	Search	29
4.6.2	Library revision	30
4.7	The setup does not encode BToM	31
4.8	Success criteria	32
5	Results	34
5.1	The library	34
5.2	The order of arrival	37
6	Limitations and Further Work	39
6.1	Ablation runs	39
6.2	Coverage and the budget frontier	39

6.3	Persistent models, interaction, and observation	40
6.4	The missing arrow $C \times C \rightarrow U$	40
6.5	Metarepresentation	41
6.6	Recursive mindreading	41
7	Conclusion	43
8.	Appendix	45
8.1.	The task corpus	45
8.2.	The primitive catalogue	52
8.3.	The final libraries	54
8.4.	The per-family description-length census	58
8.5.	The interpreters	58
	Bibliography	60

Chapter 1

Theory of Mind and its Acquisition

1.1 The nature and scope of Theory of Mind

The ice cream peddler’s scratchy jingle got louder still as he brought his cart to a stop nearby. A child, hitherto engrossed in sandbox amusements, abandoned her playmates abruptly. She darted across the playground, hurtling at full speed towards a gaggle of parents. Absorbing the impact of a 20 kg cruise missile, the target continued his conversation unfazed by the girl now tugging at his jeans. Only when he deigned to look down at the child did he shriek and go pale. But the girl barely had time to react, because just then she heard her name shouted from the other side of the playground. She turned to its source and ran towards a similar-looking man whose outstretched arms held two ice cream cones.

You chuckle at the scene, observing from a nearby park bench. The girl, wanting ice cream, pestered a man she thought was her father. The man, undoubtedly accustomed to his own nagging kids, ignored the girl thinking it was his own child. Descriptions like these, in terms of desires, beliefs, and intentions, seem trivial and obvious. But there’s something miraculous about them. How are you able to peer into other peoples’ minds? Mental content is unobservable; all you see is a sequence of physical actions. And yet humans, even infants, are experts at inferring mental content rapidly and accurately.

Humans have the cognitive ability to impute mental states to others: to attribute a belief, a desire, or an intention to another person. This is the capacity for “theory of mind” (ToM), also known as “commonsense psychology”, “naive psychology”, “folk psychology”, “mindreading”, “mentalizing” (Margolis et al., 2012). We’ll primarily use the term Theory of Mind (ToM), to emphasize that this capacity is a *theory* proper in that it enables behavioral prediction by positing states that are not directly observable (Gopnik & Wellman, 1992).

The capacity for ToM is integral to our humanity: to cooperate with someone you have to understand what they think and what they want, to decode someone’s words you have to begin from what they are trying to do with them. Humans make these inferences so naturally that the capacity can look like no achievement at all. We eagerly take the intentional stance towards entities in our environment (Dennett, 2002) and attribute mental states to them.

The mental states we attribute include perceptual states, bodily feelings, emotional states, and propositional attitudes such as beliefs, desires, hopes and intentions (Margolis et al., 2012). This thesis is concerned with the attribution of propositional attitudes, particularly belief. A propositional attitude relates an agent to some content (“she believes that *m*”). So to attribute a belief to an agent

is to attribute to her a model m of the world w (where m may differ from w), and then to predict that she will behave according to her own private model m rather than according to the actual world w (Crane, 2015; Perner, 1991). In this thesis we study how the *structure* of this attribution function might be acquired. How do humans, even at a young age, come to have this ability?

1.2 Experimental Evidence: Developmental Milestones

ToM doesn't arrive all at once, but in pieces over roughly four years and in a roughly fixed order (Beaudoin et al., 2020; Coşkun et al., 2020; Rakoczy, 2022; Wellman, 2018; Wellman & Liu, 2004). Long before children can say anything about minds, they parse motion as goal-directed. Woodward habituated 5- to 9-month-olds to a hand reaching for one of two objects and then swapped the objects' positions. Infants looked longer when the hand reached to the old location for a new object than when it reached to a new location for the old object (Woodward, 1998). So the infant encodes the goal object, not the trajectory.

Gergely et al. showed that, more than object-directed, the interpretation is even efficiency-sensitive: 12-month-olds expect an agent to take the most efficient means its situation affords, and are surprised when it does not (Gergely et al., 1995). Gergely and Csibra call this schema the *teleological stance* and distinguish it from Dennett's intentional stance (Csibra & Gergely, 2007; Gergely & Csibra, 2003). So before the first birthday, an infant can read bodies as taking efficient means to ends. This is *not* ToM. There's no attribution of mental states, it's just an extensional story.

The jump to ToM involves a transition from reality-centered to belief-centered prediction. In the *false-belief* task, a child watches Sally put her marble in a basket and leaves. While Sally's away, Anne moves the marble to the box. When Sally returns, the experimenter asks the child to predict where Sally will look for the marble (Baron-Cohen et al., 1985; Wimmer & Perner, 1983). Though the observing child knows the marble is actually in the box, the child must predict Sally's behavior according to *her* (false) belief, not according to the *child's* (true) belief. Children under about 3.5 years consistently fail, and (tellingly) their scores don't improve when the scenario is made more explicit.

The 3.5-year age threshold is robust across task variations, countries and manipulations, aggregated over 178 studies (Wellman et al., 2001). However it's been challenged by studies seemingly showing implicit false-belief ability around 1.5 years (Buttelmann et al., 2009; Onishi & Baillargeon, 2005; Southgate et al., 2007). But the findings have replicated badly (Barone et al., 2019; Dörrenberg et al., 2018; Kulke et al., 2018) and even granting them, they might be accounted for by early automatic tracking abilities (Apperly & Butterfill, 2009; Butterfill & Apperly, 2013). For the purposes of this thesis, we take the ToM scale (Wellman et al., 2001; Wellman & Liu, 2004) as our explanandum, which demonstrates that children perform progressively better on more complex ToM tasks as they mature.

1.3 What a good account of ToM acquisition ought to explain

We seek an explanation for how ToM is acquired. To evaluate competing theories, we'll set out some of the features of ToM that must be explained by an adequate account of its acquisition.

For one, mental content isn't in the data. Take for example someone trying to learn an intuitive physics by watching a ball's motion. The learner can simply extract regularities from what she sees, because a ball's position and velocity are recoverable from a sequence of images. A learner of ToM (intuitive psychology) doesn't have that luxury. What is available to the observer is a trajectory of bodies through space, but what has to be acquired is a vocabulary of attitudes that appear nowhere in the trajectory (Churchland, 1981; Polivy, n.d.). So an acquisition procedure which works by finding structure in the observable variables can't yield ToM.

Not being in the data, attributions are severely underdetermined by behavior. An agent walking across the room to the cupboard can be explained as wanting the chocolate and believing it is there, or as wanting to check whether it has been moved and believing it is there, or as wanting to be seen approaching the cupboard, and so on without end. Any behaviour is compatible with indefinitely many belief-desire pairs and no amount of further observation narrows the field to one, similarly to Quine on translation (Quine, 2013).¹ Yet humans select a fitting belief-desire explanation quickly, confidently, and on very little data. An acquisition theory should explain what does the selecting.

The underdetermination problem presupposes a learner who already has the concepts and must choose among ascriptions built from them. But a learner who lacks the concept *believes* cannot make even a wrong attribution. So the acquisition problem isn't just curve-fitting or parameter estimation within a given theory. It requires constructing the theory's terms, positing unobservable entities to explain observable behavior. Fodor argued that no learning process could deliver a concept the learner did not in some sense already have (Fodor, 1992; 1998), and Carey's work on conceptual change tries to answer Fodor's challenge (Carey, 2011). An account of acquisition must admit novel conceptual terms and explain how they come to exist.

And these terms should compose and embed. Since "believes" takes any proposition as an argument, the set of available attributions is unbounded: one can entertain attributions never encountered in the data, including attributions of beliefs one takes to be false and beliefs about things that do not exist. The operator also embeds in itself, which enables second-order mentalizing (Perner & Wimmer, 1985). And the attributions are systematic: anyone who can represent that Sally believes the marble is in the basket can also represent that Anne believes it is in the box (Fodor & Pylyshyn, 1988). The terms get their content from their role in a network of inferences rather than from links to observations one at a time. So a good acquisition theory must offer a representational medium in which mental content composes, embeds, and licenses inference.

Finally, we saw in Section 1.2 that goal attribution is in place before the first birthday while explicit false-belief prediction is not reliable until around four. A good account of ToM acquisition should explain this long delay. Also, the later competences look like assemblies of the earlier ones: belief attribution appears to be goal attribution plus subjective perceptual access. So in a good account, this ordering would be explained by the acquisition mechanism itself.

1.4 Theoretical perspectives on ToM acquisition

We now survey the standing accounts of how theory of mind is acquired and evaluate them according to the ToM features (Section 1.3). Nativism, empiricism, and constructivism are general positions in

¹This is a special case of the fact that data does not select a hypothesis without a prior (Goodman, 1983).

the debate about cognitive development, which we'll consider specifically in the ToM context, while simulationism is specific to ToM.

1.4.1 Nativism and core knowledge

The nativist holds that ToM is a built-in system that matures over time, so it's not acquired from data in any interesting sense. Nativist positions are mainly motivated by the poverty of stimulus argument (PoS): the data available to the child is too sparse and too degenerate to fix the target by general learning, so the constraints that do fix it must be innate and domain-specific (Berwick et al., 2011; Chomsky, 1975; 2014). Rather than answer the difficult question of how a learner comes to have terms for unobservables, the nativist denies the premise that these terms are learned.

Leslie argues for a theory of mind module (ToMM): a natively endowed, domain-specific system for interpreting the behaviour of intentional agents, which introduces attitude concepts such as *believes* and deploys a "proprietary representational system" for ascribing them to agents (Leslie, 1987; 1994). Autism studies are held as evidence that ToM is a dedicated system, since they seemingly show it can be selectively impaired (Baron-Cohen et al., 1985). Fodor argues that the child's theory of mind is not learned, because there is no coherent story on which a concept like belief could be learned (Fodor, 1992), so the productivity and systematicity of ToM are properties of a representational format that was there from the start. There's also the "core knowledge" position, which posits a small stock of encapsulated systems that give the child a skeletal domain-specific ontology from which they elaborate to more complex concepts (Spelke, 2022; Spelke & Kinzler, 2007).

The almost three year developmental delay from goal attribution to false-belief prediction is a challenge for nativists, since they must explain why an innate capacity isn't manifested until later in development. A ToMM account can appeal to maturation of the module, or argue that task performance demands mask underlying native competence (Leslie, 2005; Scholl & Leslie, 2001). But these concessions accommodate the developmental data by auxiliary hypotheses. What would be better is a theory that itself predicts staggered developmental results.

And the early findings don't show as much as they are taken to. Nativists often point to studies showing even young infants have abstract domain knowledge (Wellman & Gelman, 1992) as evidence for innateness. But such evidence doesn't by itself favor nativism, since the infant could just as well have acquired it (early) through learning rather than having it innately.

1.4.2 Empiricism, association, and cultural learning

The empiricist holds that theory of mind is learned like anything else: through experience, from statistical regularities, by bottom-up association. But what's learned isn't the structure of ToM, like attitude concepts or a domain-specific representational system (the nativist would just bring up PoS, that there's not enough data to learn that structure). For the empiricist, that structure is just how researchers model ToM at a high level.

In a connectionist implementation knowledge of ToM is distributed over the weights of a network. So concepts are patterns of activation rather than symbols, and structure in behaviour is emergent, it's not represented explicitly in the system (McClelland, 2010; Rumelhart, 1986). Connectionist networks have even been shown to account for false belief without using symbolic representation (Berthiaume et al., 2013).

There’s also Heyes’ non-connectionist position which takes that ToM is a cognitive gadget, i.e. a culturally inherited skill assembled out of general-purpose learning mechanisms and transmitted through social interaction and talk about the mind (rather than a genetically inherited instinct) (Heyes, 2018; Heyes & Frith, 2014). Ruffman argues in a similar spirit that the developmental data can be accounted for without crediting children with belief concepts at all, by low-level and statistical means (Ruffman, 2014).

Empiricist accounts are well-suited to explaining how ToM emerges even though mental content isn’t in the data. But these explanations often deny that the structure of ToM is acquired at all, instead they just show how the learner acquires useful associations from data: it learns a mapping. And then the empiricist can’t account for systematicity, since there’s no representational medium with constituent structure (Fodor & Pylyshyn, 1988).

An empiricist theory would predict that development of competencies scales with frequency of exposure to that data, but that doesn’t explain the developmental order (Section 1.2). No empiricist implementation shows how a data frequency difference leads to a two year false belief delay. Also, the later competences appear to presuppose the earlier ones rather than to be acquired alongside them as separate regularities learned from their own statistics. The cultural-learning empiricist could attribute the competency ordering to the ordering of the input (Heyes & Frith, 2014), but that’s a hypothesis about the input rather than a mechanism, and it predicts that the developmental sequence should vary between children more than it appears to (Wellman & Liu, 2004).

1.4.3 Simulationism

Simulationism is different than all the other positions because it doesn’t see ToM as a body of knowledge that needs to come from somewhere, native or learned. There is no concept “believes” to be constructed, since ToM is simply the process of putting oneself in another’s shoes.

On this account, to “attribute belief” isn’t attribution. Rather the observer feeds pretend inputs (which stand in for the target’s beliefs and desires) to her own decision-making machinery, and reads the output off as her prediction about the target (Goldman, 2006; Gordon, 1986; Heal, 2004). The pretend beliefs are usable since they’re of the same kind as the target’s real beliefs they stand in for (Goldman, 2006).

The problem is that, though the simulated run does yield a decision, nothing makes that decision *Sally’s* rather than a stray conclusion of the attributor’s own. Nor does anything in it keep the pretend inputs from contaminating what the attributor takes the world to be. So simulationists need to give a containing structure with explicit boundary conditions to the output of one’s personal decision-making machinery (Nichols & Stich, 2003).

Also, simulationism doesn’t explain the false-belief delay (Section 1.2). If belief attribution simply uses the observer’s own planner, simulationism should predict that false-belief competency arrives soon after the development of the (teleological) planner around the first birthday (Gergely et al., 1995), but it’s delayed by more than two years. And any simulationist explanation of the delay would be extra-theoretical. So there are few “pure” simulationists (Goldman, 2006; Nichols & Stich, 2003).

1.4.4 Rational constructivism and the “theory theory”

The constructivist holds that the child is analogous to a scientist (Downes, 1999; Gopnik, 2012): she constructs intuitive theories of mind and world and revises them in light of evidence, by something like the process that drives scientific progress (Gopnik, 1997; Gopnik & Wellman, 1992; Wellman et al., 1990). So the child uses domain-general statistical machinery to construct domain-specific structured knowledge. Infants are indeed sensitive to sampling and to statistical evidence in ways that indicate inference to hidden structure, including psychological structure (Wellman et al., 2016; Xu, 2019; Xu & Kushnir, 2013), and children’s exploratory behaviour is well described as evidence-gathering aimed at resolving uncertainty about which theory is right (Schulz, 2012).

On this account the transition from failure to success in the false-belief task is analyzed as an evidence-driven shift between two theories. The infant starts with a simple model, but after encountering evidence inexplicable by that model switches to a more complex one which offers greater explanatory resources (Goodman et al., 2006).

Rational constructivists specify that Bayesian inference is the formal mechanism of theory change (Gopnik & Wellman, 2012; Newcombe, 2011; Xu et al., 2012). Structured, abstract knowledge, embedded in a probabilistic framework, can be acquired from sparse and noisy data (Gopnik & Tenenbaum, 2007; Griffiths et al., 2010; 2011; Perfors et al., 2010; Ullman & Tenenbaum, n.d.; Xu, 2019). Theory formation generally has been modeled as probabilistic inference over systems of related concepts (Goodman et al., 2011; Kemp et al., 2010), and theory change has been modelled as stochastic search through a space of theories generated by a grammar (Ullman et al., 2010).

In our analysis, the rational constructivist’s theory theory comes closest to explaining the features of ToM laid out in Section 1.3. Theories have compositional embeddable terms are constructed cumulatively and learned from data, so the order of acquisition is a consequence of what has already been built.

The position is a middle way between the first two we surveyed. Nativism posits domain-specific logical symbolic mechanisms, whereas empiricism posits domain-general probabilistic statistical mechanisms. Rational constructivism instead asks “how can domain-general mechanisms of learning and representation build domain-specific systems of knowledge? How can structured symbolic knowledge be acquired through statistical learning?” (Griffiths et al., 2024; Tenenbaum et al., 2011).

Like the empiricist, the constructivist believes that ToM is learned from data. But what is learned is the actual terms and structure of ToM itself, the body that the nativist claims is innate and cannot be learned. So the rational constructivist’s position hinges on the assertion that structured symbolic representations “need not be rigid, static, hard-wired, or brittle” but “can grow dynamically and robustly in response to the sparse, noisy data of experience” (Tenenbaum et al., 2011). Absent a mechanism for growing these theories, the nativist can say that no constructive process exists so that what looks like theory construction is really the maturation of a native module.

In this thesis we respond to the nativist’s claim by counterexample. We propose a model of a process that searches a space of structured hypotheses, selects ones that fit the data on principled grounds, and from correct hypotheses learns new theoretical terms which encode the structure of ToM.

Chapter 2

Learning as Bayesian Program Synthesis

2.1 The computational-level problem

The learner observes a visual scene depicting a trajectory of bodies through space, and outputs an explanation of the process that generated the trajectory. The output process is formulated in terms of mental states (beliefs, desires, intentions) that don't appear in the data. The trajectory underdetermines the process, since any behaviour is compatible with indefinitely many belief-desire pairs (Section 1.3). So no amount of further observation eliminates all but one.

The problem is of a familiar form: given data, entertain hypotheses about the process that generated it, assign each a prior plausibility, and revise those plausibilities as new data comes in. *Bayes' rule* is the known answer to the normative question of how such revision ought to proceed. So the computational-level claim (Marr, 2010) is that acquiring a theory of mind is the problem of finding the best explanation of observed behaviour, where the best explanation takes into account both its prior plausibility and how well it fits the data.

2.2 Bayesian inference: the logic of learning

Say a learner observes data d , a trajectory of bodies through space. A learner's task is to choose among possible explanations $\mathcal{H}$ a hypothesis h that best explains the trajectory. The task is to choose the most plausible hypothesis given the data, i.e. to find the hypothesis whose posterior probability $P(h|d)$ is highest. In calculating this quantity, the learner should weigh two things. First, the prior probability $P(h)$: how plausible was hypothesis h before any data d was observed at all? Second, the likelihood $P(d|h)$: how well does hypothesis h account for the data d , now that data is available? Bayes' rule specifies how to combine these considerations:

$$P(h|d) = \frac{P(d|h)P(h)}{\sum_{h' \in \mathcal{H}} P(d|h')P(h')}$$

Since the denominator is a normalising constant that doesn't depend on h , it can be ignored for the purpose of ranking hypotheses against each other. So we rewrite the equation as the proportion

$$P(h|d) \propto P(d|h) \cdot P(h)$$

The posterior probability $P(h|d)$ of a hypothesis h given data d is proportional to the likelihood $P(d|h)$ of the data d given hypothesis h times the prior probability $P(h)$ of hypothesis h before data d was observed. Bayes' rule is a theorem of the probability calculus: given the basic commitment to representing degrees of belief as probabilities, it's the unique coherent way to revise them in light

of evidence (Jaynes & Bretthorst, 2003; MacKay, 2019). Having stated the computational problem the mind faces, Bayes’ rule specifies the ideal solution.²

The prior $P(h)$ expresses the a priori plausibility of a hypothesis, i.e. how likely the learner takes h to be, independent of the current observations. So the prior encodes the learner’s background knowledge. For example a hypothesis that is outright impossible has prior $P(h) = 0$ so it’s excluded from $\mathcal{H}$ and no data observation can increase its posterior probability. The prior determines which explanations are more natural, so we can start to generalize from among the indefinitely many hypotheses consistent with the data.

Anything can be a prior, since it’s just what encodes the learner’s background knowledge, assumptions, and preferences. In this thesis the prior we’ll focus on is the preference for simplicity. Hypotheses that are short, that reuse existing structure rather than stipulating new structure, will have higher priors than hypotheses that are long and idiosyncratic. This assumption is supported by accounts of human rule-based concept learning (Chater & Vitányi, 2003; Goodman et al., 2008), and by the finding that the difficulty of learning a Boolean concept tracks the length of its shortest logical expression (Feldman, 2000). In Section 2.3 we’ll specify how the simplicity of a hypothesis is calculated, after we’ve said what hypotheses are made of.

The likelihood $P(d|h)$ expresses how probable the observed data would be if the hypothesis were true. In our setting a candidate hypothesis either explains the observed behaviour (i.e. it reproduces the trajectory exactly) or it does not. So the likelihood is all-or-nothing: the data have probability zero $P(d|h) = 0$ if they disagree with the hypothesis, and constant probability otherwise (Goodman et al., 2008). The binary likelihood can only ever eliminate hypotheses (likelihood $P(d|h) = 0$ in the product yields posterior $P(h|d) = 0$). So once a set of hypotheses all fit the data perfectly (only those with constant likelihood remain), the likelihood can’t distinguish between the remaining hypotheses and their ranking is decided entirely by the prior.

2.3 Programs as the hypothesis space

Bayes’ rule doesn’t specify what we should define our hypothesis space over, so we can choose the best medium for representing our hypotheses. Since we choose simplicity as our prior, hypotheses should have parts that can be counted so their relative size and complexity can be compared. And a hypothesis must be executable, since the indicator likelihood asks whether running a hypothesis reproduces the observed trajectory. A finite learner must have unbounded and systematic capacity (Section 1.3), so the hypotheses must be generated by rules of combination (Fodor & Pylyshyn, 1988). And also the hypothesis space must admit new terms, since we want to model how a learner acquires the terms that denote mental content (attitudes) even though they don’t appear in the data.

These requirements rule out connectionist representations (Section 1.4.2) where a hypothesis is a weight vector or a point in a high-dimensional parameter space (Tenenbaum et al., 2011). Though they’re efficient to update, they lack a natural notion of countable constituent parts so we can’t

²To be clear this claim is at the computational level, not at the algorithmic level. Nobody is saying that a child computes posterior probabilities, or represents the hypothesis space $\mathcal{H}$ explicitly, etc. The claim is that the child’s behavior approximates the solution to an inference problem whose ideal form is described by Bayes’ rule. It’s not an algorithmic-level claim about the procedure by which the child does this, though we’ll later introduce an algorithm for approximating this inference.

compare their relative sizes. What we need is a structured symbolic representation like a grammar, predicate logic, relational schemas, or functional programs (Griffiths et al., 2024; Tenenbaum et al., 2011). What we need is a language of thought (Fodor, 2010), where mental representations are symbolic objects combined by syntactic rules and evaluated as a function of their parts (Bruner et al., 1986; Carey, 2011; Siskind, n.d.).

Programs are the natural medium for our hypothesis representation. They are compositional, since the value of a composite expression is a function of the values of its parts. And a program admits any expression of the right type in any argument position of that type. They are executable, which is what the indicator likelihood requires. And they encode procedural and declarative knowledge in the same format (Rule, n.d.).

Problem solving has been treated as symbol manipulation since the advent of the computational theory of mind which views the mind as a compact program that mirrors the structure of the world (Baum, 2004; Goodman et al., 2014; Newell et al., 1958). A symbolic medium also leaves the learner free to adopt new vocabulary beyond what the modeler supplied (Gopnik & Wellman, 2012). And furthermore, if knowledge is programs then learning is program induction (Flener & Schmid, 2008; Gulwani et al., 2017; Schmid et al., n.d.) which we’ll soon make use of.

We adopt the λ -calculus as our program representation formalism, since it’s the standard choice for models of learning across domains (Piantadosi et al., n.d.; 2012; Schmidt, n.d.; Zettlemoyer & Collins, n.d.). Most importantly it gives us λ -abstraction as the operation of taking a recurring piece of structure and giving it a name. So this way a learner can come to possess a term that names something that isn’t observable in the data, since the new term’s content is fixed entirely by what it does in combination with the other terms.³

Since our programs are composed of symbols of a grammar, a distribution over a grammar’s productions induces a distribution over expressions. Since the probability of an expression is the product of the probabilities of the choices made in generating it, longer expressions (involving more choices) get less probability mass. The cost of writing a message in a code optimal for that distribution is the negative log of its probability (Shannon, 1948), so h ’s description length under the given grammar is $DL(h) = -\log P(h)$. So the prior is $P(h) \propto e^{-DL(h)}$. And we have an indicator likelihood for whether executing h reproduces the observed behavior. Substituting into Bayes’ rule, the posterior is computed as

$$P(h|d) \propto e^{-DL(h)} \cdot \mathbb{1}[h \text{ solves } d]$$

So maximizing the posterior is just minimizing the description length of correct programs. The learner should adopt the program which has the shortest description length of all programs that solve the data (i.e. that reproduce the trajectory). This is basically the theory theory (Section 1.4.4): the learner should prefer the simplest theory that accounts for the evidence.

³Again, we don’t claim the mind implements a λ -calculus interpreter, only that its ToM competence is well described as inference over hypotheses that compose, execute, embed, and can be measured for size (which are all properties the λ -calculus representation supplies).

2.4 Program induction and the search problem

The trouble is that the optimization problem of searching through program space for one that fits the data is about as badly behaved as an optimization can be⁴. The space is infinite, since a grammar whose productions can nest inside one another generates unboundedly many expressions, and the number of expressions of a given size grows exponentially in that size. There’s no gradient to follow, because programs are discrete: there’s no meaningful sense in which one is slightly perturbed into another. And the binary likelihood doesn’t give partial credit, so a program that differs from a correct one by a single symbol receives exactly the same zero-likelihood as a wrong program picked at random. So the search is blind: there’s no signal anywhere in the space to hill-climb on, and no way to tell that one is getting warm.

To make the search less inefficient at least we can enumerate hypotheses in decreasing order of prior probability, checking each against the data until one fits, so given enough time we find a solution program. Since $P(h) \propto e^{-\text{DL}(h)}$, enumerating every program of description length at most ℓ takes time on the order of e^ℓ . The time to find a solution is exponential in the description length of that solution under the current library.

Note that in Section 2.3 we introduced description length as a way of ranking hypotheses, but this way it also determines which hypotheses are even reachable at all. The prior is simultaneously a measure of plausibility and a budget. A concept’s being expressible in the language doesn’t guarantee that it’s learnable in the language within a reasonable budget.

Description length is relative to the library available, so the same program may be long under one library and short under another that happens to contain the right symbols. So facing the search problem, we should just change the library so the target gets shorter. If we add symbols to the library that are useful for the domain, programs that were previously out of reach will fall within the budget since they can now be expressed more succinctly. This doesn’t make the problem tractable in general, but it might make particular programs tractable when they weren’t before.

2.5 Learning the language itself

If we’re permitted to adjust the library whenever the search is going badly, then the modeler can make anything learnable by just adding a primitive that does the work. What we need is an account on which changing the library is not a device the modeler applies from outside but something the learner does as part of the same inference it was already doing.

In the setup (Section 2.2) the learner is given a hypothesis space $\mathcal{H}$ and a prior $P(h)$ which is the type-uniform distribution over the library’s symbols. So revising the prior is just changing which symbols the learner has. When a structure recurs across the solutions found for a domain, that recurrence is evidence that the structure is useful. It can then be named, added to the library, and thereafter costs a single symbol wherever it appears. Programs that previously were prohibitively long can now be expressed more succinctly, since the complexity of the often-used structure is abstracted into a new named symbol.

⁴Here we start to move from the computational level toward the algorithmic one, i.e. the procedure in which the computational problem is solved (Marr, 2010; Tenenbaum et al., 2011)

Adding a symbol — revising the library — follows a similar inferential principle as in Section 2.3 but applied a step further out. Given a corpus of observations $D = \{d_1, \dots, d_n\}$, the goal is to find the best library $\mathcal{L}$ for that corpus. The posterior $P(\mathcal{L}|D)$ is given by:

$$P(\mathcal{L}|D) \propto P(\mathcal{L}) \cdot \prod_{i=1}^n \sum_h P(d_i|h) \cdot P(h|\mathcal{L})$$

A library’s prior $P(\mathcal{L})$ is determined by its simplicity: a library with fewer and less-complex symbols has higher prior probability. Without this complexity cost the best library is always the one carrying a dedicated symbol for every task in the corpus, so it overfits to the specific corpus and won’t generalize to similar corpora of the same domain.

The product runs over the corpus of n tasks $D = \{d_1, \dots, d_n\}$. For a given task d_i and a given hypothesis h , the likelihood $P(d_i|h)$ indicates whether executing program h correctly produces d_i (zero otherwise). And $P(h|\mathcal{L})$ is the description length prior (Section 2.3) relative to a library, so $e^{-\text{DL}(h|\mathcal{L})}$ is the cost of writing h in $\mathcal{L}$. Then the program h is marginalized away, since a library just needs to make an adequate hypothesis available.

Because the likelihood is an indicator, every h that fails to reproduce d_i contributes nothing to the sum. So the inner sum collapses to the total prior mass the language places on programs that solve the task, $\sum_{h \text{ solves } d_i} P(h|\mathcal{L})$. So the equation says that a good library is one under which solutions are cheap, and cheap for every task in the corpus at once. This is the posterior over libraries inferred in (Ellis et al., 2020).

Learning the library rather than a hypothesis within it has been done for intuitive theories generated by a probabilistic grammar (Ullman et al., 2010), and for program fragments transferred from solved problems to unsolved ones (Liang et al., 2010). DreamCoder adds that what gets learned is a library of named λ -expressions, so that revising the library means acquiring new concepts rather than redistributing probability among symbols the learner already had (Ellis et al., 2020).

But settling a library is an even worse optimization problem than program space search (Section 2.4), since the space of candidate libraries is larger than the space of hypotheses under any one of them. DreamCoder approximates it by maximizing a lower bound on the posterior (Ellis et al., 2020). In rounds, it alternates solving a batch of tasks under a fixed library, then revising the library in light of what the solutions turn out to have in common (Dechter, 2013). An abstraction found in a later round may be defined using abstractions found in earlier ones, so the library accumulates in layers.

2.6 Our claim

In our picture of learning, “strong but domain-general inference and representational resources are aided by weaker, domain-specific perceptual input analyzers” (Goodman et al., 2011). Of course something must be built-in, so our disagreement with the nativist is about what that must be. On the modular view what’s innate includes the attitude concepts themselves, a proprietary representational format for ascribing them, and the agent-directed inferential machinery that operates on it (Leslie, 1994). On our view what’s innate is a capacity to represent structured hypotheses, a preference for

short ones, a procedure for searching, and a stock of low-level domain-general primitive operations. So anything specific to the mental is constructed.

The construction of a conceptual vocabulary is cumulative (Carey, 2011; Piantadosi et al., 2012). An abstraction acquired at one stage becomes a term in the language in which the next stage’s hypotheses are written, so at each stage more complex hypotheses can more easily be posited. This correctly predicts the developmental progression (Section 1.2): ToM is a layered vocabulary, and its components should become available in an order fixed by what each one is built out of.

Goal-directedness is available early because it’s an abstraction over a great many observed pursuits. Belief is available late because it’s an abstraction over goal-directed pursuit together with the ability to posit counterfactual “what-if”s, so it can’t be assembled before its parts exist. The developmental delay (Section 1.3) is a consequence of the compositional structure of what’s being learned, so it’s directly explained by the theory theory ((Palmarini et al., 2024)).

We give the argument: if a learner equipped only with domain-general primitives and a preference for short programs comes to posit belief-like structure, then positing such structure is something a learner *can* in principle do rather than something it must be given. So a counterexample blocks the nativist’s inference from “no mechanism has been named” to “no mechanism exists”. Whether human learners use this mechanism, or some other one that solves the same problem, is a further question that this thesis does not settle.

Chapter 3

Bayesian Theory of Mind

3.1 Agent as planner

In this chapter we first claim that Bayesian Theory of Mind (BToM) is the best available computational account of the mature ToM capacity described in Chapter 1. BToM formalises the intentional stance as inverse planning, and its posteriors quantitatively track human judgements. We then claim that in BToM the structure of belief attribution is stipulated by the modeler rather than inferred from the data. Of course this isn't a shortcoming of BToM, since BToM isn't a model of how ToM is acquired but a model of mature ToM competence.

To adopt the intentional stance towards a system is to treat it as a rational agent. First attribute to it the beliefs and desires it ought to have (given its situation and history), then predict its behavior by working out what a rational agent would do (Dennett, 2002). People adopt the intentional stance even towards two triangles and a circle moving about a rectangle, seeing them as chasing, hiding, bullying, and escaping each other (Heider & Simmel, 1944). The intentional stance is a strategy that gets used because it regularly yields predictive success (Dennett, 2002).

Bayesian Theory of Mind (BToM) gives a computational account of this strategy, modeling the agent as a planner. The agent occupies one of a set of states, has available in each state a set of actions, and the actions move it between states according to a transition function. The planner takes as input a utility function and a model of the environment, both defined over the agent's state space, and returns the actions the agent should perform to maximize utility (Baker, 2012; Baker et al., 2009; Lake et al., 2016). Even twelve-month-old infants encode actions as directed at a goal state, and expect an agent to use the most efficient means available to Section 1.2 (Csibra & Gergely, 2007; Gergely et al., 1995; Gergely & Csibra, 2003).

3.2 Inverse planning

To plan what to do, a rational agent computes in the forward direction: given the model of the world m and a goal g , find the optimal behavior (the trajectory t) for achieving that goal. So the agent computes $t = \text{optimize}(g, m)$, where “optimize” is a planning algorithm that finds the best path to g which is available on m .

An observer watching an agent, trying to explain the agent's behavior, needs to compute in the backward direction: given the world model m and the agent's trajectory t , find the goal that, assuming a rational agent, would yield such a behavior. Say the observer hypothesizes that the agent's goal is g' . To evaluate the hypothesis, the observer can simulate what the agent would have done if its goal were indeed g' . The observer calculates the resulting trajectory as $t' = \text{optimize}(g', m)$ since the observer assumes the agent is rational. Then if the simulated trajectory t' matches the actual trajectory t , it means the hypothesis that g' is the agent's goal (that $g = g'$) is correct.

The process is analogous to modeling intuitive physics by running a physics engine forward under different settings and scoring the outcomes (Battaglia et al., 2013; Lake et al., 2016).

Using Bayes’ rule (Section 2.2), the learner finds the hypothesized goal that best explains the agent’s behavior. Writing g for the agent’s goal, m for its model of the environment, and t for the observed agent’s trajectory, the posterior is calculated as

$$P(g, m \mid t) \propto P(t \mid g, m) \cdot P(g, m)$$

where the prior $P(g, m)$ encodes what the observer expects agents to want before seeing anything, and the likelihood $P(t \mid g, m)$ encodes the probability that an approximately rational agent with goal g and model m would move as observed (which is supplied by the planner) (Baker et al., 2009). It’s been shown experimentally that people’s predictions on behavioral stimuli are close to the model posteriors of this approximately rational inference mechanism (Baker et al., 2009; 2017; Jara-Ettinger & Rubio-Fernandez, 2021).

3.3 Belief as an input

The posterior is joint over (g, m) , the agent’s goal g and a model m of the environment in which it navigates. But m doesn’t have to be an accurate model of the actual state of affairs (the observer’s world), it could be any possible world model. The observer can supply a counterfactual model m' that diverges from the actual state of affairs, and then run the same posterior computation to see how the agent would behave if it were navigating in the counterfactual world m' . Then the same computation that yields sensible behavior from an accurate model would yield mistaken behavior from the inaccurate one.

This is belief attribution in essence, attributing a counterfactual world to an agent such that it acts according to the counterfactual model as its own private world. So we can ask: given an (irrational) trajectory, which model of the environment would’ve made those actions rational? Take the false-belief task (Section 1.2): Sally puts her marble in the basket and leaves; in her absence the marble is moved to the box; and when she returns she searches the basket (Baron-Cohen et al., 1985; Wimmer & Perner, 1983). Evaluated against the actual (observer’s) world m , Sally searching the basket is irrational since the marble is in the box. But evaluated against her private model she had when she left m' (that the marble is in the basket), her searching the basket is optimal. An observer that can represent the divergence of worlds can predict that Sally will look in the wrong location, and even which wrong location she’ll look in (Baker et al., 2017; Goodman et al., 2006).

To calculate the likelihood, the observer needs to simulate the predicted trajectory. The agent’s private model m' is a counterfactual derived from the observers as $m' = \text{derive}(m)$, where “derive” modifies m to account for the agent’s false belief which the observer imputes. In the Sally–Anne example, in the observer’s model m (the real world) the marble is in the box, whereas Sally’s model m' (her private world) is the same as m in every respect except that the marble is in the basket.

The observer’s then needs to run the agent’s planner on its private model, to yield its trajectory (behavior) given that model. Rather than running $t' = \text{optimize}(g', m)$ on her own model m (as in Section 3.2), the observer runs $t' = \text{optimize}(g', m')$ on the agent’s private model m' . So t' is the agent’s predicted trajectory, given that it pursues goal g' along the most optimal path afforded by

counterfactual model m' . Then t' can be compared to t , to see whether the predicted goal g' and model m' predict that a rational agent will take trajectory t' .

The “optimize” planner is the same one from Section 3.1. It’s indifferent to whether its input is the real world model m or an incorrect world model m' . Most importantly, it’s indifferent to whether its input model is *attributed to an agent* or not. All it does is run a search algorithm on input data. The model is executed and evaluated the same way. So there is nothing belief-specific about the planner.

3.4 What is stipulated by the BToM modeler

BToM is a model of the ToM capacity of a mature adult (Baker et al., 2009; 2017). It’s a model of the computational problem that a person who has ToM is solving when they are trying to understand a person’s behavior. So the ToM capacity is specified by the modeler. Inference in a BToM is parameter estimation within a fixed theory, but we’re interested in construction of the theory’s terms. We’re interested in how the capacity might be acquired. In Chapter 4 we’ll introduce a model that (we’ll argue) doesn’t have the mature ToM capacity which is stipulated by the modeler of a BToM model. To argue that these features aren’t stipulated, we’ll state them explicitly here.

Most obviously BToM assumes the rationality principle, that the agent plans approximately optimally toward its goal given its model Section 3.1. We assume this as well in our model, because efficiency-sensitive goal-directed navigation is not ToM Section 1.2. But a lot of other feature of the BToM model are so basic that they’re easily taken for granted. For example in a BToM model, the visual scene is often already decomposed into an agent that has a goal g and a belief m , with the modeler already stipulating the roles that different entities play. The planner takes them as separate arguments that vary and can be inferred independently of each other.

A BToM model stipulates the attitudes that the learner searches over, for example the space of goals. The posterior over goals is inferred but the space it ranges over (and the prior over the space) is given. Baker et al. compare several goal priors (Baker et al., 2009), because the modeler has to specify the space. The model space that the posterior ranges over, the fact that the planner’s input model m' is permitted to diverge from the observer’s m , is similarly stipulated by the modeler. And the modeler has to specify the divergences between m and m' . The BToM model infers which particular model m' the agent holds, but the range its drawn from is fixed in advance. That a belief could be false is not itself in question.

These features are still stipulated even in extensions of the BToM framework that are themselves compositional. Shum et al. adapt inverse planning to groups of agents, where an observer has to recover not just what each agent wants but how the agents stand to one another. They do this by composing a small set of team operators recursively, so that arbitrary cooperative and competitive structures are built up from a compact set of building blocks (Shum et al., 2019). The operators are compositional, but the operators themselves are still specified by the modeler. The posterior locates a point among their compositions, rather than arriving at the operators themselves. Compositionality in the hypothesis space is not the same thing as constructing the vocabulary the hypotheses are written in.

The BToM modeler’s stipulations are not hypotheses about any particular scene. But they are hypotheses about what the hypotheses about scenes *are like*. That behaviour is generated by an agent, that the agent has a goal and a model, that the model may diverge from the world, that the divergence is drawn from some particular range. These stipulations are the learner’s language, the vocabulary in which a particular attribution gets stated at all. BToM stipulates that vocabulary and uses it to run inference. The question of this thesis is whether the vocabulary and structure itself can be inferred. Our question is whether the language, given to the framework by its modeler, is the kind of thing a learner could arrive at by the same inference it runs on everything else. So the stipulated features of BToM constrain what we’ll be permitted to endow our model with, in that we can’t grant it these same features if we want them to be discovered.

Chapter 4.

The Model

4.1. The learning problem

In our model the learner is an observer shown tasks from a corpus D : sets of visual scenes that unfold over discrete time steps on a small grid (Section 4.2). For each task its goal is to recover the process that generated what it saw, by positing candidate hypotheses until one reproduces the scenes. A hypothesis is a program, a composition of symbols drawn from the current library, and the learner starts from an initial library $\mathcal{L}_0$ of primitives (Section 4.4). Learning alternates two phases (Section 4.6). In *search*, the learner finds for each task the shortest program under the current library that reproduces its scenes. In *compression*, it finds syntactic structure shared across the found programs. Commonly shared structure is named as a new abstraction and is added to the library, yielding $\mathcal{L}_{i+1}$. Each abstraction turns what was a deep subtree into a single symbol, so compositions that were unreachable in one round come within the search budget of the next, and the learner’s library grows iteratively more abstract, useful, and domain-specific.

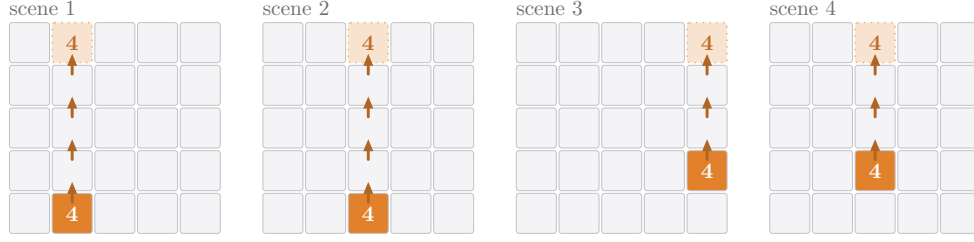
We ask whether belief attribution (Section 3.3) is among the abstractions such a learner invents, given a domain-general initial library $\mathcal{L}_0$ and task corpus D . In this context, to attribute a belief is to open a private copy of the world, modify it, run the planner against the copy rather than the world, and modify the real world according to the result computed in the copy (Section 4.5). If the learner adds an abstraction with this structure to the library, then belief attribution is learnable by the same domain-general induction the learner applies to everything else. So the demonstration would be a counterexample to the claim that belief attribution must be acquired through a native domain-specific module (Leslie, 1987).

4.2. The task corpus

Our corpus consists of 316 tasks in 25 “families”, groups of tasks that are all governed by a common underlying process. Of these 168 tasks in five families can only be solved by belief attribution, and 148 can be solved extensionally. The search doesn’t know this distinction about tasks of course, it treats them all alike.

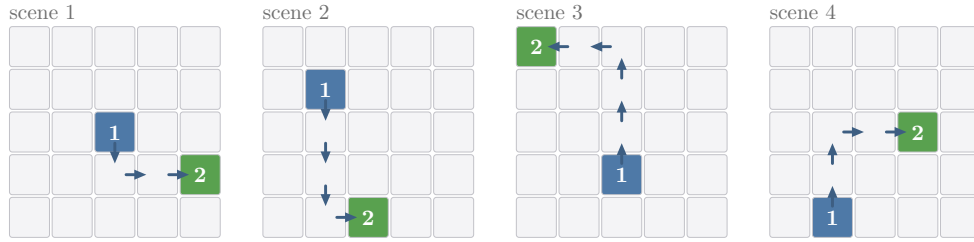
Most tasks are *trajectory tasks*. A trajectory is a sequence of 2d grids over the time dimension, that depicts some dynamic at play (e.g. a body that falls, an agent that approaches something, etc.). A task consists of k trajectories that all depict the same dynamic but vary in setup (where the entities start, how long the scene takes, what else is lying on the grid, etc). Given a task, the learner’s goal is to find a transition function that, for any trajectory in the task, returns the next grid for any grid in the sequence. The transition function should be a general explanation of the underlying dynamic, invariant to initial conditions.

For example, the task below is an instance of the *constant-movement* task family. It consists of four scenes that each depict the value 4 moving up. In the figure the grids of each timestep are collapsed into one, so each arrow indicates one timestep forward.



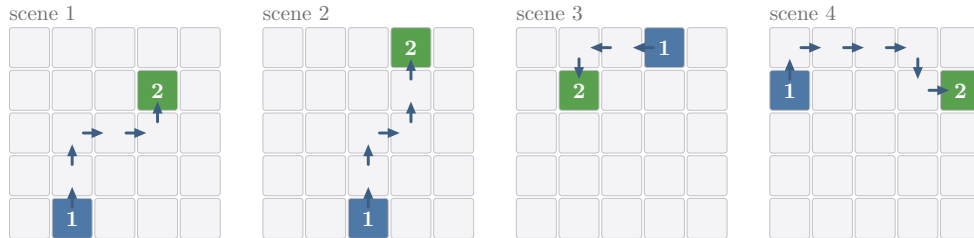
A solution to this constant-movement task would be a program that encodes “*at each time step, value 4 moves to the adjacent cell above it*”. Such a program solves the task because for each scene, given that scene’s starting positions, the program reproduces the rest of the trajectory.

The following task is an instance of a *goal-directed movement* task. Value 1 is an agent moving towards its goal 2. Throughout this chapter, if we say “agent” and “goal” it’s just for didactic reasons, since that’s the role these values are playing. But they’re not encoded as “agent” and “goal”, to the observer they’re just the values 1 and 2 on the grid.

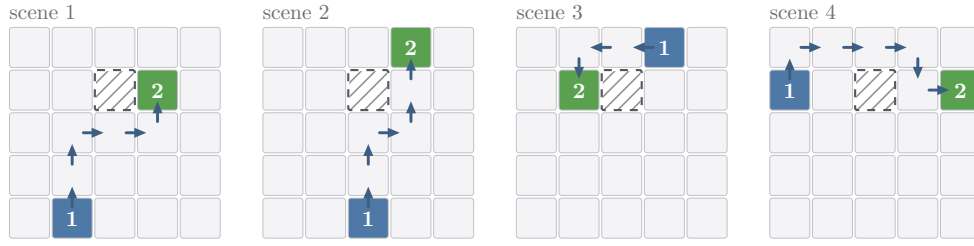


A solution to it would be a program that encodes “*at each time step, the value 1 moves to the adjacent cell closest to value 2*”. The solution involves the optimizing planner (Section 3.1), but *does not* require attributing a belief. It’s a purely teleological description.

The following is an instance of an *obstacle* task. The same solution program for the above goal-directed task reproduces the first three scenes. But notice that in scene 4, value 1 takes a suboptimal path to value 2 (it should just go straight to the right but it goes up first). So we need a program that accounts for this, so it solves all four scenes.



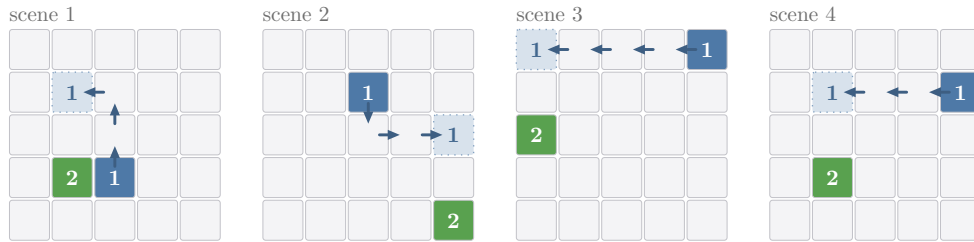
The origin (0,0) is the top left cell. A solution might encode “*at each time step, value 1 moves to the adjacent cell closest to value 2, in a world where an impassable wall sits at cell (2,1)*”. The wall isn’t part of the world that the observer sees. The observer hypothesizes that there’s a wall there, to make sense of the agent’s suboptimal movement.



The four scenes of a task are different “observations” of the same “world”, governed by the same underlying process, so a solution to the task is a single program that solves all four. So even though only scene 4 needs the wall for its trajectory to make sense, the program posits a wall in the other scenes as well. The scene 4 trajectory could just as well be explained by a wall at (1,1) or (3,1), but those would interfere with the trajectories of the other scenes which pass through those points. So given the constraints, a wall at (2,1) is the solution.

The solution to the above task may seem like ToM or belief attribution, but it is not. The observer does indeed postulate a world model that differs from her own, but she doesn’t *attribute* that divergent model to a particular agent. The solution “*at each time step, value 1 moves to the adjacent cell closest to value 2, in a world where an impassable wall sits at cell (2,1)*” is a hypothesis about an agent’s behavior in a counterfactual model. This is distinct from a hypothesis about an agent’s behavior in *that agent’s* counterfactual model. It’s not that the agent takes a wall to be at (2,1), it’s the *observer* who takes it to be there. So there’s no belief attributed to the agent.

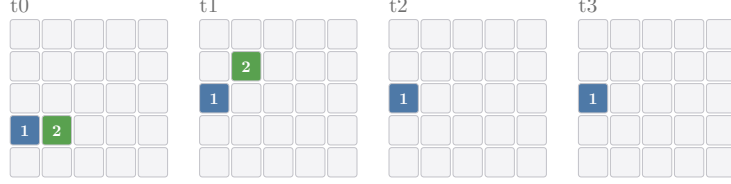
These task families, among others, are “non-mental”: they don’t require belief attribution to be solved. The task families we care about are the mental ones, those that can only be solved by attributing a belief to an agent; since the mental state they all require attributing is a belief, we call them the *belief families* from here on. The following task is one of them, an instance of the *goal-displacement* family. This is our version of the Sally–Anne task. The light blue cell is value 1’s final location at the end of the scene. So to the observer, value 1 moves towards an empty space which is consistently two cells above value 2.



The task would be solved by a program that encodes “*at each time step, value 1 moves to the adjacent cell closest to the cell two cells above value 2*”. But this program isn’t expressible in our language. The planner is an expression of the capacity for taking the teleological stance (Gergely & Csibra, 2003), a goal-directed efficiency-sensitive measure. So it can only aim at an object on the grid, not at a location.⁵

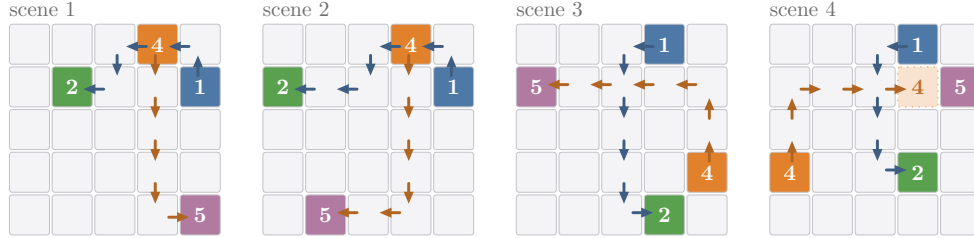
⁵Coordinates do exist as terminals, but they are arguments to the two grid edits (stamping a wall, clearing a cell) and nothing carries them to a utility, nor is there any arithmetic on them with which to form “two above” in the first place. And the agent can’t be given something to aim at, since the only primitive that writes at a named coordinate writes a wall, walls are impassable, and a wall would show up in the frames.

Instead, a candidate program that is expressible by our language is “*at each time step, move value 2 two cells up, then value 1 moves to the adjacent cell closest to value 2*”. But that doesn’t reproduce the scenes correctly. Below is a frame-by-frame of the result of running the program on the first frame of scene 1. At t_1 value 2’s jumps two cells up, at t_2 it’s off the top edge and gone from the grid, so value 1 stays put since no adjacent cell movement decreases distance to value 2. The problem is that the program needs to relocate 2 but only for the purposes of serving as a reference for 1’s movement. The program shouldn’t *actually* be moving 2 in the real world.



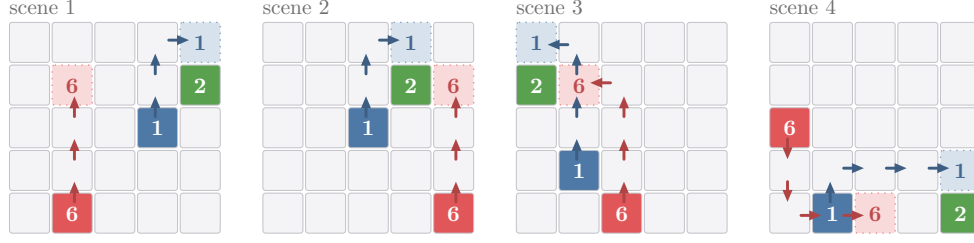
What we need is a program that encodes “*at each time step, value 1 moves to the adjacent cell closest to 2, given that value 1 is navigating according to its own (false) world model in which value 2 is two cells above where it actually is.*” This is the belief attribution of BToM: a hypothesis that posits that an agent is navigating according to their own private model of the world, which may differ from the real world. The program posits the agent is navigating by its private model, but doesn’t render the model as the real world. The private model affects the agent, but only the agent: the goal’s position is unaffected.

Another belief family are the *witness* tasks. In the instance below, value 1 bends around cell (3,1) to navigate to value 2, while value 4 navigates to value 5 without avoiding (3,1).



A hypothesis that posits a wall at (3,1) won’t do, since value 4 wouldn’t be able to pass through that cell if there were a wall there. So to explain these scenes, a program needs to differentiate between the model by which value 1 navigates and the model by which value 4 navigates. It needs to attribute a counterfactual model to value 1, in which there’s a wall at (3,1). But to value 4 the observer doesn’t need to attribute any belief, it can just explain its behavior with respect to the real world that the observer sees. So a solution program would have to encode “*at each time step, value 4 moves towards value 5. and value 1 moves towards value 2 according to a model that has a wall at (3,1).*”

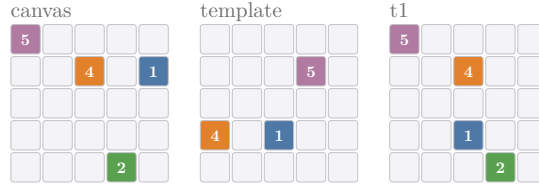
A third belief family, the *two-observer* family, combines the goal displacement with a second agent. In the instance below both value 1 and value 6 move towards value 2, but value 6 walks to value 2 itself, while value 1 walks to the empty cell one above value 2 and stops there.



As in the goal-displacement family, value 1’s endpoint is explained by a private model in which value 2 sits one cell above where it actually is. Value 6’s path needs no such model, it’s explained by the world as the observer sees it. So a solution program has to encode “*at each time step, value 6 moves towards value 2, and value 1 moves towards value 2 according to its own model in which value 2 is one cell above its actual position.*” Both agents pursue the same value, and the false belief about it must be attributed to exactly one of them — the same selectivity the witness family demands, but now with a single shared goal.

All of the task families so far, belief or not, are *trajectory tasks* where for each scene the input is a starting grid and the goal is to find a transition function. A second, smaller group of tasks are *template tasks*. A template task’s input is a pair of grids (canvas, template), and the goal is to find a program mapping the canvas and template pair to an output grid. The format is loosely inspired by the ARC-AGI-1 corpus [CITE].

For example, below is an instance of a *registration* task. A program that solves this task encodes “*change value 1’s position on the canvas to its position specified by the template*”.



Registration comes with a handful of variant families that turn the same alignment job around. In *perception* tasks it’s the template that is updated to record where the value stands on the canvas. In *drifting registration* the alignment is carried out inside a moving image. We refer to them collectively as the *registration families*. The template families are in the corpus because they populate the pair interface from the non-mental side. To entertain two models of the world at once is not specifically a mental capacity. It’s the domain general capacity to entertain a counterfactual, to “what-if”. A program that consumes a canvas–template pair is aligning two images it was given, not holding or attributing a belief.

The template families are not the only non-mental claim on two-grid structure. Three *picture-processing* trajectory families (*overlay*, *underlay* and *comet*) depict a value that smears across the grid rather than moving through it. The cells it has passed through are never cleared, so each frame is the previous frame with the move unioned onto it. Reproducing that takes the same copy-and-collapse machinery as everything else in this section, but in those tasks it’s used to paint motion blur, just graphics, with nothing mental in it.

We created the families in the corpus to claim all the pair combinators (Section 4.4), to show that the primitives of our library are domain-general. See the full corpus of task families in Section 8.1.

4.3. Program space: type-directed enumeration

A hypothesis is a program, so the hypothesis space is the space of programs generated by type-directed enumeration. G is the type of 2d grids of cell values, the frames of a scene. Write G^* for stacks of grids, i.e. a scene (among the four in a task). A transition function alters a grid, so it's of type $G \rightarrow G$. The cell values on the grid are members of the ground set $V = \{0, \dots, 9\}$. Value 3 is reserved as an impassable wall, while the other values are interpreted as agent or goal or any other entity. The grid coordinates are $C = \{0, \dots, 4\}$, a row or column index, from which a position in $C \times C$ is built. The remaining sets are D , the four movement directions, and $U \subseteq (C \times C \rightarrow \mathbb{R})$, the positional utilities that the planner scores (Section 4.4).

Everything else the language names is a function between these. The ones relevant to our argument are the functions into and out of the pair type $G \times G$, which serves as the (world, model) channel pair. Whether a program opens a private second channel is a matter of which of those types its nodes inhabit, so a (world, model) representation is a structural property of the program. The fixed pair type does carry the commitment that a learner holding a private representation holds exactly one; generalizing the arity is left as further work.

A program is checked by running it through an interpreter (Section 8.5). The solution to a trajectory task (Section 4.2) is a program of type $G \rightarrow G$, a transition function that given the task's initial grid at step i should return the next grid at step $i + 1$. The interpreter iterates the program $G \rightarrow G$ on the task's initial grid G , yielding the stack of grids G^* which can be compared to the target scene. The interpreter's state at any step is just a single grid G , not a (world, model) pair $G \times G$, and certainly not a registry of agents, goals, or entities. No pair is handed from one step to the next: a program that opens a second channel (one that may diverge from the world) must build it, use it, and collapse it again inside that step.

A template task starts from a pair in $G \times G$, a canvas and a constant external template, so its solution is a *commit* $G \times G \rightarrow G$, one of the collapses of Section 4.4. The commit answers what each frame should do with the template; the grid it returns is the next frame's canvas, while the template is re-paired with it unchanged. Since a type-directed enumeration yields programs of one type only, trajectory tasks ($G \rightarrow G$ -rooted) and template tasks ($G \times G \rightarrow G$ -rooted) are searched separately (Section 4.6.1). Though compression pools the solutions of both (Section 4.6.2).

4.4. The primitive library

Our learner is initialized with an initial library $\mathcal{L}_0$ of primitives. We designed our library so it would be domain-general, with the primitives low-level enough that they are each used in a variety of task families, combined in different ways with various other primitives. None of our primitives encode the structure of ToM, a belief attribution, or any other mental operation. For every primitive we added to the library we also included another that plays its complementary role. This way we have a point of comparison to show that the learner chooses certain primitives for a program but not others. The full list of primitives is in Section 8.2.

To interface with elements on a grid, our library includes primitives for changing the values on grids. The *planner*, of type $U \times V \rightarrow (G \rightarrow G)$, is the one from Section 3.2: given a utility and a cell value it returns a transition function that at each time step moves that value one step toward

the position maximising the utility. The two utilities it can be handed, both of type $V \rightarrow U$, are minus and plus the distance to the nearest cell of a named value, so the same planner approaches a value under the one and flees it under the other. Beside it are the movements and edits, all of them transition functions once their arguments are supplied: a constant *step*, of type $V \times D \rightarrow (G \rightarrow G)$, which moves a named value one cell in a named direction; *stamping a wall* at a position and *clearing* a single cell, both of type $C \times C \rightarrow (G \rightarrow G)$; *erasing*, of type $V \rightarrow (G \rightarrow G)$, which scrubs every cell of a named value from the grid; and *composition*, of type $(G \rightarrow G) \times (G \rightarrow G) \rightarrow (G \rightarrow G)$, which runs one transition function and then another. The terminals are the four directions, the ten cell values, and the five coordinates, a pair of which names a position.

Since our programs involve pairs of grids $G \times G$, we need primitives for accessing values in them. A *reader*, of type $V \rightarrow (G \rightarrow C \times C)$, gives the position of a named value in a grid. A *writer*, of type $V \rightarrow (G \times (C \times C) \rightarrow G)$, gives the grid with that value moved to a given position. A third primitive, of type $(G \rightarrow C \times C) \times (G \times (C \times C) \rightarrow G) \rightarrow (G \times G \rightarrow G)$, joins a reader to a writer. It reads a position off the second channel and imposes it on the first.

The rest of the primitives are the *pair combinators*: standard, content-blind operations on a pair of grids that never look inside either of them. There are four jobs to be done with a pair, and a group of combinators for each: building a pair out of a grid, transforming a pair, collapsing a pair back to a grid, and reading and writing the positions a collapse goes off of. In what follows w and m are grids, f and f' transition functions, and v a cell value.

Two combinators *build* a pair, both of type $G \rightarrow G \times G$. The *copy* $w \mapsto (w, w)$ puts a grid in both channels. Its complement $w \mapsto (w, \mathbf{0})$ pairs the grid with an empty one instead, so that the second channel starts as scratch rather than as a copy of the world.

Four combinators *transform* a pair, all of type $G \times G \rightarrow G \times G$. Three of them *map* a transition function over the channels, leaving the pair a pair: $(w, m) \mapsto (f(w), m)$ applies it to the first channel, $(w, m) \mapsto (w, f(m))$ to the second, and $(w, m) \mapsto (f(w), f'(m))$ applies one to each. Each is a separate primitive. The fourth is the *swap* $(w, m) \mapsto (m, w)$, which exchanges the channels, so that anything defined on one of them can be turned into a version acting on the other by swapping before and after.

Several combinators *collapse* a pair to a single grid, of type $G \times G \rightarrow G$. The two projections, $(w, m) \mapsto w$ and $(w, m) \mapsto m$, return one channel and discard the other. Two more union the channels cell by cell, differing only in which of them wins where they disagree. The last two are *scope* collapses. One moves every value the two channels share to the position it holds in the second. The other, of type $V \rightarrow (G \times G \rightarrow G)$, does the same for every shared value but one named exception. So they both adopt the second channel's value but they differ in how much of it to adopt.

Each of these arrows needs its own composer, since the composition of the grid layer joins two transition functions only. Three are given, one for each arrow that needs one. A builder followed by a transform, of type $(G \rightarrow G \times G) \times (G \times G \rightarrow G \times G) \rightarrow (G \rightarrow G \times G)$, extends a builder. A builder followed by a collapse, of type $(G \rightarrow G \times G) \times (G \times G \rightarrow G) \rightarrow (G \rightarrow G)$, is the composer that returns to $G \rightarrow G$; since the interpreter only ever iterates a transition function, a program that opens a second channel must close it again within the same time step, and this is what closes it. A

transform followed by a collapse, of type $(G \times G \rightarrow G \times G) \times (G \times G \rightarrow G) \rightarrow (G \times G \rightarrow G)$, is the only one that acts on a pair the program did not itself build, which is what a template task’s given pair calls for.

Nothing in any of these primitives distinguishes a world from a model. The pair is symmetric: both factors are grids, every map is present in both directions, and no primitive marks either channel as a particular agent’s private representation. Which channel is the world and which is a belief is settled only by how a program assembles these pieces together, which is found by search.

4.5. Composing belief attribution

In Section 3.3 we presented belief attribution as a two-step computation. First, the observer must *derive* a counterfactual model which deviates from her own according to the agent’s false belief $m' = \text{derive}(m)$. Then the observer *runs* the agent’s movement policy on the counterfactual model $t' = \text{optimize}(g, m')$ to yield the agent’s simulated trajectory which is optimal w.r.t the agent’s world m' .

This structure can be constructed by composing the $G \times G$ primitives from Section 4.4. In our setup only the *run* step is done by a primitive, the planner, which builds the agent’s movement policy. Nothing in the library opens a private copy, so the *derive* step of creating and modifying a copy of the grid must be discovered by the learner.

Our setup also has a third step, *commit*, which writes the result of the private computation back to the world. This step doesn’t have a counterpart in Section 3.3. BToM’s observer runs the planner inside herself and gets the trajectory t' back as a value, which she then compares against the observed t . What happened in the private model never has to re-enter the world, because the observer is the one holding both. Our programs have no such vantage point. A hypothesis is a transition function $G \rightarrow G$ and the interpreter renders one grid per step (Section 4.3), so a move made on a private copy is invisible unless something explicitly writes it into the grid that gets rendered. Without a commit, nothing an agent does on its own model ever reaches the predicted trajectory outputted by the program.

Another reason why our setup has a *commit* step is that the commit is where attribution lives. In BToM, that g and m' are agent a ’s is settled by the modeler before the computation starts: they are handed to the planner as a ’s arguments, and the trajectory that comes back is a ’s by declaration. Our learner has no place to make that declaration, since no type says whose a grid is and the interpreter cannot tell a private copy from any other grid (Section 4.7). So if a counterfactual grid is to be *somebody*’s rather than merely counterfactual, the program has to specify that. The commit is where the program can encode which unique entity’s movement is determined by the counterfactual model, i.e. the entity who has that model as their private model. The commit separates between positing a counterfactual world and positing it as the world a particular agent is acting on.

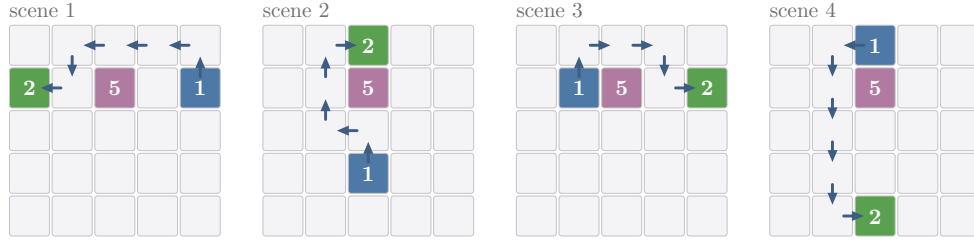
So in our setup there are two target structures needed for recreating the belief-attribution structure. We call them the *single-value commit* and the *derive-and-commit frame*. The *single-value commit* is of type $V \rightarrow (G \times G \rightarrow G)$. Given a value, it reads where that value sits in the private copy and moves the same value in the world to that position, so exactly one value’s position is published and everything else stands as the world had it.

The derive is the transition function $(G \rightarrow G)$ that is applied to the believing agent’s private copy. The *derive-and-commit frame* is of type $(G \rightarrow G) \times (G \times G \rightarrow G) \rightarrow (G \rightarrow G)$. Handed a derive and a commit, it copies the world into a pair, applies the derive to the second channel to get the private copy, and gives the pair to the commit, which collapses it back to one grid. The private copy exists only for the duration of that call.

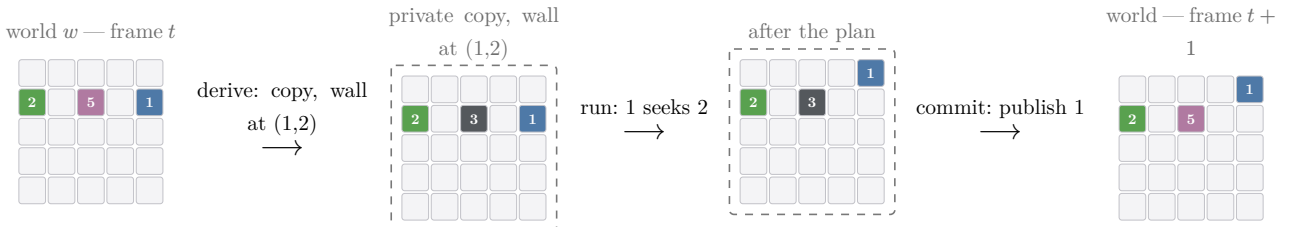
Belief is the composition of the two, with the derive applying the agent’s error and the commit doing the attribution. The derive-and-commit frame $(G \rightarrow G) \times (G \times G \rightarrow G) \rightarrow (G \rightarrow G)$ applied to the derive $G \rightarrow G$ and commit $G \times G \rightarrow G$ yields a transition function $G \rightarrow G$.

We’ll walk through the process of belief attribution through derive-run-commit on a *false-wall* task. This is a simpler variant of the witness task (Section 4.2): rather than a witness whose path crosses the location of the agent’s falsely believed wall, the bystander remains static at that location the entire run. In our experimental setup, 3 is an impassable wall, and any value other than 3 is a generic value through which another value can pass.

Take scene 1 below. An agent without a false belief that there’s a wall would just take the direct path to 2. So value 1 must believe there’s a wall somewhere between it and 2. The only available cell for a wall there is at (2,1), since value 1’s trajectories pass through (1,1) and (3,1) in scenes 2 and 3 respectively.



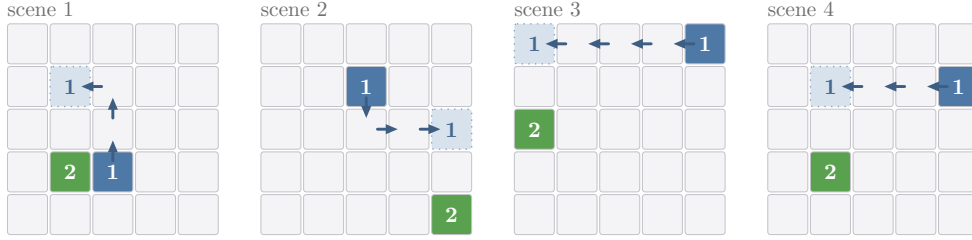
We’ll now show frame-by-frame the derive-run-commit process, for scene 1. Observer starts with its initial world model. It then *derives* a copy of the grid and applies a transformation (placing a wall) to it. So the private copy now has wall 3 where the bystander 5 was before. The observer then *runs* the planner on the private copy, which yields agent 1’s next position according to that copy. Now that the observer has agent 1’s position, it *commits* that value to the world grid, and only that value. So by the second timestep, nothing in the world is changed except for agent 1’s position, which was changed according to its private model. So the wall never reaches the shared world, and the bystander 5 remains untouched.



Note that the agent value a occupies three slots in that program: the slot that says whose move is planned over the privately walled grid, the slot that says whose position is read off the copy, and the slot that says whose position is written back to the world. So to find this compound — a composition of several symbols that is not itself in the library — the search needs to independently fill the

argument slots of the three primitives with the same value placeholder. We call that coincidence the *agency signature*, the structural mark that separates an attribution (a private grid that is *somebody's*) from a mere counterfactual world.

The Sally-Anne false belief task is easily solved by an observer whose library carries belief attribution as a single symbol. As stated in Section 4.2, the task below is solved by a program that encodes “*at each time step, value 1 moves to the adjacent cell closest to 2, given that value 1 is navigating according to its own (false) world model in which value 2 is two cells above where it actually is.*”



We can formulate that program purely in terms of grid operations, as:

at each time step, *derive* a model of the grid by moving value 2 two steps up in a copy of the grid. Then in the model, *run* the planner to find the adjacent cell closest to value 2 and move value 1 to that cell. Then *commit* value 1’s location from the model to the world, i.e. in the world move value 1’s position based on its position in the model, and don’t move anything else.

4.6. Inference

Our procedure runs in alternating rounds of exploration and compression, the loop of (Dechter, 2013) as in DreamCoder (Ellis et al., 2020). The exploration phase performs per-task inference, enumerating programs under the current library for the tasks not yet solved (Section 2.4). The compression phase revises the library to compress the programs found (Section 2.5).

DreamCoder’s third phase trains a neural recognition model that reads a task and emits a task-conditional prior $Q(\cdot | x)$, so that enumeration proceeds under a Q biased toward the primitives that task is likely to need. We don’t implement it, because of the difference in scale between our domain and the DreamCoder experiments.

The amortized inference that the recognition model offers is essential when the branching factor is large and the loop is long. DreamCoder works with libraries of hundreds of symbols over dozens of iterations. Ours is a small library run for a few rounds with fewer than 50 symbols. There is no long tail of iterations across which a learned proposal could amortize anything.

4.6.1. Search

The exploration phase finds a hypothesis that correctly explains a scene, by enumerating candidate programs in decreasing order of prior probability and checking if the program produces the scene. A hypothesis is a program (Section 2.3), represented as a typed expression tree over the current library (Section 4.3). Since enumeration in decreasing prior probability is enumeration in increasing description length, the search walks outward in expanding bands of cost (expanding $\sim e^\ell$, see

Section 2.4). The first program it finds that reproduces the data is the shortest one that does, which is the maximum a posteriori hypothesis (h^* , see Section 2.3). The enumeration runs under a time budget, and a hypothesis whose cost band is not reached before the budget expires is simply not found (Section 2.4).

The library fixes the prior. The prior $P(h) \propto e^{-\text{DL}(h)}$ (see Section 2.3) is realised as a distribution over the library’s symbols, under which a program’s cost is the summed $-\log p$ of its tokens, the occurrences of library symbols at its nodes. Symbols are grouped by return type, and a symbol of type τ is priced at $-\ln$ of the number of symbols sharing τ , i.e. a type-uniform distribution. So the types give the branching factor at each argument slot, and the cost of each choice at that slot. Enumeration is type-directed: an argument slot draws only from symbols whose type matches, so no ill-typed tree is ever built. (Note that one exception to the type-uniform distribution is that a cell value which is visible in the task’s first frame costs nothing, since naming what is already on display is not a hypothesis about anything. But a placed wall is full-price, since it’s positing something that isn’t there).

A candidate program solves a task only if the interpreter, started from each scene’s own first frame, reproduces that scene’s every frame exactly, for all k scenes (Section 4.2). So the indicator likelihood $\mathbb{1}[h \text{ solves } d]$ (Section 2.2) is evaluated against all k scenes.

4.6.2. Library revision

Compression returns a set of abstractions together with the corpus rewritten to use them. Each abstraction is registered as a new symbol in the library. So what was a subtree found in round i becomes a single symbol in round $i + 1$, so the next enumeration stage can reach deeper into program space (Section 2.5) and the learner bootstraps more complex programs.

Where exploration minimizes description length one hypothesis at a time against a library held fixed, compression minimises the total description length. It minimizes the length of the library itself plus the summed cost of every solved program written in it (Ellis et al., 2020), the posterior over languages (Section 2.5).

$$\text{DL}(\text{library}) + \sum_{\text{solutions}} \text{DL}(\text{program} \mid \text{library})$$

When a syntactic structure is common to a lot of programs, it gets added to the library as a new abstraction. So the description length of the solutions shrinks, since rewriting the solutions replaces the common subtrees with the abstraction’s symbol (so $\sum_{\text{sols}} \text{DL}(\text{program} \mid \text{library})$ shrinks). But the library is now larger, and since a symbol’s probability falls as the number of symbols competing with it rises, every program written in the enlarged library gets a little more expensive (so $\text{DL}(\text{library})$ grows). This is the tradeoff that needs to be made when deciding whether to abstract some shared structure. The two summands ensure that an abstraction is kept only when the tokens it saves across the whole corpus outweigh the price of adding it. So we’re less likely to abstract incidental structure, and more likely to abstract structure that’s relevant to the corpus.

DreamCoder looks for abstractions bottom-up, enumerating refactorings of each found program and intersecting them (Ellis et al., 2020). Instead we delegate the search to the top-down Stitch (Bowers et al., 2023) library-learning system, which searches the space of abstractions directly

and so never has to materialize every rewrite. Also, note that enumeration searches by root type (Section 4.3), so solutions to trajectory tasks (transition functions $G \rightarrow G$) and template tasks (commits $G \times G \rightarrow G$) are searched for separately. This is just an implementation necessity which doesn't affect the overall outcome, since compression searches for common structure among the full pool of solutions together regardless of root type.

4.7. The setup does not encode BToM

The goal of our setup is to recover the belief attribution structure from Chapter 3 without stipulating it ourselves. A natural objection to our results would be we've merely re-encoded Chapter 3's stipulations in a different notation so the learner trivially "finds" them. To pre-empt this, we'll argue that no notion of belief attribution, agency, or world/model distinction is encoded in our setup.

In our primitive library we give the pair type $G \times G$ and the pair combinators over it. The pair is symmetric and content-free: the copy produces (w, w) , and nothing in it says the second copy is a *model* or that anyone *holds* it. Belief is assembled by composition: acting on the second channel rather than the first, committing directly rather than conjugating through the swap, publishing one named value rather than every value the channels share. Each of those choices has its opposite in the library, and each opposite is exercised by some non-mental family in the same run. So the medium is neutral, and any choice of particular abstractions is made because they are useful to the corpus.

The interpreter does not supply ToM either. In BToM the divergent model is an input to a planner that the observer runs inside itself, so the observer already distinguishes between its own model and any counterfactual model. Our interpreter has one grid of state, no second channel to hand anybody, and no mode distinguishing evaluating a program from evaluating a program-as-an-agent-would. A private copy exists only because a program built it.

One might then argue more strongly that the ability to build a copy of the world that differs from the world just *is* the representational capacity of ToM. But it is not. Building a world state that differs from the actual one is just counterfactual reasoning, the capacity to "what-if", to consider what would be the case if things were otherwise. It's a domain-general ability, and infants have this ability way before false-belief attribution.

ToM is an abstraction that *uses* the counterfactual capacity, by *attributing* the divergent world to a particular agent, so that it is *that agent's* world. That attribution is the conjunct the agency signature encodes (Section 4.5). The primitives don't supply that signature: the derive-and-commit frame can just as well derive a private copy, compute over it, and publish an unrelated value rather than the agent's movement.

In BToM the modeler must also fix the ways in which a private model may diverge from the observer's, and the scenarios that produce the divergence. In our setup there's no range to fix, since the derive slot accepts any transition function $G \rightarrow G$. So the different task families that require false belief attribution differ in what the belief is about.

BToM's goal space, and the prior on it, are given by the modeler (Baker et al., 2009). In our setup, a goal is whichever cell value happens to fill the argument of the attracting utility, drawn from the same ten integer terminals that supply agent identifiers and step magnitudes. There is no goal type and no goal prior. A value seeks something because a symbol landed in that slot. Even the

polarity is a search choice, since the repelling utility sits alongside the attracting one, so maximising utility flees a value as readily as it approaches one. The only prior over any of it is the type-uniform distribution, which prices a goal value by the same rule that prices a direction or a composition. The same applies to “agents”, which have no special type or status. We just call a value an “agent” when it’s used in a program in a way that it plays the role of an agent, like having a belief attributed to it.

Rationality is the one stipulation of BToM that our learner also receives, in the form of a planning primitive. We grant it because it’s not a mental operation: the teleological stance relates an action, a goal state and the constraints of a situation without attributing any representation to the agent (Gergely & Csibra, 2003). Planning is in place well before the first birthday and some three years before anything recognizable as false-belief attribution (Section 3.1, Section 1.2) (Gergely et al., 1995).

The transition function given by the planner $U \times V \rightarrow (G \rightarrow G)$ just takes a single grid as input. It has no parameter identifying whose grid it is and no access to any other channel, it just modifies a grid like any other grid-edit primitive. It cannot represent a divergence of models because it cannot see two things to diverge, and it cannot attribute anything because it has no argument for an attributee.

Belief arises only when the surrounding program hands the planner a grid that is not the world, and then publishes only part of the result (Section 4.5). And furthermore the planner is used in a variety of task families that don’t involve belief attribution. The planner is certainly a hefty primitive, packing a full breadth-first search into one token. But this algorithmic complexity isn’t evidence that the planner is doing mental work, that it grants a theory of mind in and of itself.

In BToM the posterior ranges over goals and models (g, m) . So nothing in the framework makes a goal attribution earlier or cheaper than a belief attribution, the inference proceeds the same over both. This isn’t a problem with BToM, since it models mature ToM capacity where we can reasonably assume that the difference in processing power between goal attribution and model attribution is negligible. But for our purposes we’re interested in the developmental timeline, in explaining the fact that goal attribution arrives at least two years before belief attribution. So our setup accounts for that delay: a belief compound (Section 4.5) is a deep composition whose cost under the initial library puts it far outside any feasible budget window, and it becomes reachable only once earlier solutions have been compressed into abstractions that shorten it.

4.8. Success criteria

In our setup, successfully acquiring the ability to attribute belief means the learner adds an abstraction to its library whose body (i) opens a private channel, (ii) derives from it a world state that diverges from the actual one, (iii) evaluates an agent’s policy against that divergent state, and (iv) commits the agent’s resulting move, and nothing else of the divergence, back to the world.

Condition (iv) is what distinguishes attributing a belief from positing a counterfactual. A private copy is *somebody’s* belief only if it’s used in a way that makes sense with respect to the world and agent. Take Figure 11, where the private model shows agent 1 navigates according to a wall, which

is agent 1’s false belief. The relevant value that should be committed from that model is the agent, not the wall. The agent’s movement is caused by something false, but that movement is real.

Our language allows for several ways of demarcating which value or values get committed from the model to the real world. In the Figure 11 example, the commit is “publish value 1”, naming the one value that goes through from the model to the world. The same commit can be formulated as the complement, “publish everything except the believed content (the wall)”, holding back the false value while everything else moves to where the model has it. And the commit can be left to a default, to just publish the union of values, the values that the grids share.

For our purposes these spellings are interchangeable (mostly because our tasks are so small, in bigger tasks the difference would be meaningful). For example, the commit from Figure 11 could be rewritten as “commit everything except the wall”, since the wall is the only value in the model that is false or whose position deviates from its position in the real world.

Regardless of whether the commit is formulated as inclusion or exclusion, the program would only count towards the criterion if the value the commit takes is bound correctly. The value whose move reaches the world must be the value the policy of (iii) moves, and no false belief postulated by the derive of (ii) may reach the real world.

So which values need to be bound depends on which commit spelling the program uses. For example, if the program uses the inclusion commit “publish value 1”, then value 1 must be the value that moves according to the false belief. Similarly, if the program uses the exclusion commit “publish everything but value 3”, then value 3 must be the value that is false in the private model. A program whose commit publishes an unrelated value (or renders the private copy in its entirety including a posited wall) would just be a counterfactual world with no owner. So it would fail the condition on account of a binding mismatch (Section 4.5).

Other than the structure of the found abstractions, we are also interested in the process by which the abstractions are selected for the library and this is the most relevant factor for our original point of inquiry about the acquisition of ToM. So our success criteria also include that the abstraction must be selected by the joint compression objective of Section 4.6.2. An abstraction should be selected only if the description length it saves across the solved corpus outweighs the cost of registering it.

The components that an abstraction is assembled from must be demonstrably domain-general. They should occur across the corpus in different combinations, solving a variety of families of tasks. The belief-attribution abstractions must arrive by the bootstrapping detailed in Section 4.6. They should be out of reach under $\mathcal{L}_0$, but within reach after earlier solutions are compressed.

In Chapter 5 we report the run against these criteria, including the library it converged to and the bodies of its abstractions (Section 5.1), and the order in which those abstractions were assembled (Section 5.2).

Chapter 5

Results

We ran the learner of Chapter 4 on the corpus (Section 4.2), initialized with the primitive library (Section 4.4). We ran it until convergence, until it no longer found new solutions so there was nothing new to abstract from. It converged in four rounds, solving 280 of the 316 tasks (132 of the 168 belief tasks), and its final library contains six learned abstractions. This chapter presents those abstractions.

For each abstraction we present its type, its body, and what the transition function it returns does at each time step. Bodies are written with letters assigned by role⁶: a is an agent in V , g a goal in V , b a believed-about value in V , n a second actor in V , r, c a wall’s coordinates in C , d a direction in D , v a value in V , f a transition function in $G \rightarrow G$, k a commit in $G \times G \rightarrow G$. The run’s own symbols, argument orders and verbatim bodies are in the appendix (Table 5).

5.1 The library

Abstraction	Symbol	Type	Solves (belief / non-mental)
phantom-wall constructor	fn_9	$C \times C \times V \times V \rightarrow (G \rightarrow G)$	75 / 0
two-actor constructor	fn_7	$V \times D \times V \times V \rightarrow (G \rightarrow G)$	26 / 0
displaced-goal constructor	fn_10	$V \times D \times V \rightarrow (G \rightarrow G)$	22 / 0
open constructor	fn_6	$V \times V \times (G \rightarrow G) \times (G \times G \rightarrow G) \rightarrow (G \rightarrow G)$	12 / 0
seek block	fn_8	$V \times V \times (G \rightarrow G) \rightarrow (G \rightarrow G)$	46 / 47
agent commit	fn_11	$V \rightarrow (G \times G \rightarrow G)$	9 / 12

Table 1: The library the run converged to

For a given abstraction, the solve counts indicate how many solution programs contain the abstraction as a subtree. For example, the phantom-wall constructor appears in 75 solutions to belief tasks, but in no solutions to non-mental tasks. The full abstractions are at Table 5.

We gave each of the final abstractions a name for ease of reference. We’ll go through each of these final abstractions, showing its type and its body, both in terms of its constituent abstractions and also fully inlined. Note that we show the abstractions written as they are outputted by the run itself. We haven’t introduced these implementation-level names for the primitives and they’re not necessary because we also give the types and we’ll explain their semantics.

⁶To be clear, these “roles” are just the names we give them based on how they’re used in the given example

Open constructor — $\text{fn_6} : V \times V \times (G \rightarrow G) \times (G \times G \rightarrow G) \rightarrow (G \rightarrow G)$

```
(pipe_gpg (compose_gp dup (mapsnd (compose f (optimize (neg_dist g) a)))) k)
```

Applied to an agent a , a goal g , a derive f and a commit k , it returns the transition function: at each time step, copy the grid; on the copy, make the alteration f , then move a one step toward g ; then hand the (world, copy) pair to k , which collapses it back to one grid. This is the derive-and-commit frame of Section 4.5 with the planner already inside it: what the copy is wrong about (f) and how the copy is published back to the world (k) are both left as holes. Twelve belief solutions use the open constructor directly, with the search filling both holes.

The next three abstractions are the open constructor’s instantiations. The compression step emits each of their bodies as a call to fn_6 with the two holes filled. So we give each body in that call form first, with its expansion beneath it.

Phantom-wall constructor — $\text{fn_9} : C \times C \times V \times V \rightarrow (G \rightarrow G)$

```
(fn_6 a g (wall_at r c) sync_all)
= (pipe_gpg (compose_gp dup (mapsnd (compose (wall_at r c)
      (optimize (neg_dist g) a)))) sync_all)
```

At each time step: copy the grid; on the copy, stamp an impassable wall at (r, c) , then move a one step toward g ; then move every value the two grids share to the position the copy gives it.

The wall exists only in the copy, so it is not a shared value, and a is the only shared value the derive moves. So moving all the values moves just a and nothing else, and the wall never reaches the rendered world. This is the false-wall attribution: a detours around a wall that is not there. This abstraction solves 75 belief solutions, the most of any abstraction.

Displaced-goal constructor — $\text{fn_10} : V \times D \times V \rightarrow (G \rightarrow G)$

```
(fn_6 a b (step b d) (sync_except b))
= (pipe_gpg (compose_gp dup (mapsnd (compose (step b d)
      (optimize (neg_dist b) a)))) (sync_except b))
```

At each time step: copy the grid; on the copy, displace b one cell in direction d , then move a one step toward b ; then move every shared value *except* b to where the copy has it.

So a walks toward where b is not, a ’s move is published to the the world, and the displacement (being b ’s alone) is never published to the world. This is the Sally–Anne attribution: a acts on a false belief about where b is. The value b fills three slots at once: the value displaced on the copy, the value a ’s plan pursues, and the one value the commit withholds. This abstraction appears in twenty-two belief solutions.

Two-actor constructor — $\text{fn_7} : V \times D \times V \times V \rightarrow (G \rightarrow G)$

```
(compose (optimize (neg_dist b) n)
  (fn_6 a b (step b d) (sync_except b)))
= (compose (optimize (neg_dist b) n)
```

```
(pipe_gpg (compose_gp dup (mapsnd (compose (step b d)
      (optimize (neg_dist b) a)))) (sync_except b)))
```

At each time step, n moves one step toward b in the world itself, and then the displaced-goal constructor runs for a .

This is the displaced-goal constructor with a second actor composed in front. The `fn_6` call here is the displaced-goal constructor’s whole body. Both actors pursue the same value b ; n plans against the world as it is and a against a private copy in which b has been displaced, so the two end up at different cells. Here b fills four independently searchable slots: it’s displaced on the copy, sought by n , sought by a , and withheld by the commit. The abstraction binds one value to all four arguments.

Given the value 2, the direction up, the value 1 and the value 6, the abstraction encodes “*agent 6 navigates towards goal 2. agent 1 also does, but believes 2 is one cell up*”. Twenty-six belief solutions use this abstraction, 24 of them the two-observer family’s tasks, each solved by this one symbol applied to four values.

Seek block — $\text{fn_8} : V \times V \times (G \rightarrow G) \rightarrow (G \rightarrow G)$

```
(compose f (optimize (neg_dist g) a))
```

Make the edit f to the grid, then move a one step toward g — all of it in the world, with no copy and no commit.

This is also, symbol for symbol, what the open constructor runs on the copy (the library carries that occurrence inlined inside `fn_6` rather than as a call). This is the goal-directed movement family’s planning policy with room for one edit in front of it. This is the abstraction found from the non-mental corpus in round 1. 47 obstacle solutions run it with a real wall as the edit, and 46 belief solutions (nearly all from the witness family) run it with a phantom-wall constructor in the edit slot: the believer’s whole attribution step as the “edit”, followed by the witness’s seek on the world it leaves behind.

Agent commit — $\text{fn_11} : V \rightarrow (G \times G \rightarrow G)$

```
(register (locate v) (place v))
```

Applied to a value v , it returns a commit: read where v sits in the second grid, move v in the first grid to that position, and leave everything else as the first grid had it.

This is the single-value commit of Section 4.5, reassembled from the registration parts, with v bound twice: once in the reader and once in the writer. Twenty-one solutions use it: 9 belief solutions, where it publishes exactly the believer’s move, and 12 from the registration families, where the same abstraction does image alignment against a template.

The three constructors spell their publication three different ways, committing three different things. The agent commit commits the *agent*: publish this one value and nothing else. The phantom-wall constructor commits the *everything*: move everything the grids share, which, on a scene where the copy differs from the world only at an unshared wall and at the agent, is the same one-value transfer. The displaced-goal and two-actor constructors commit the *false-belief content*: publish

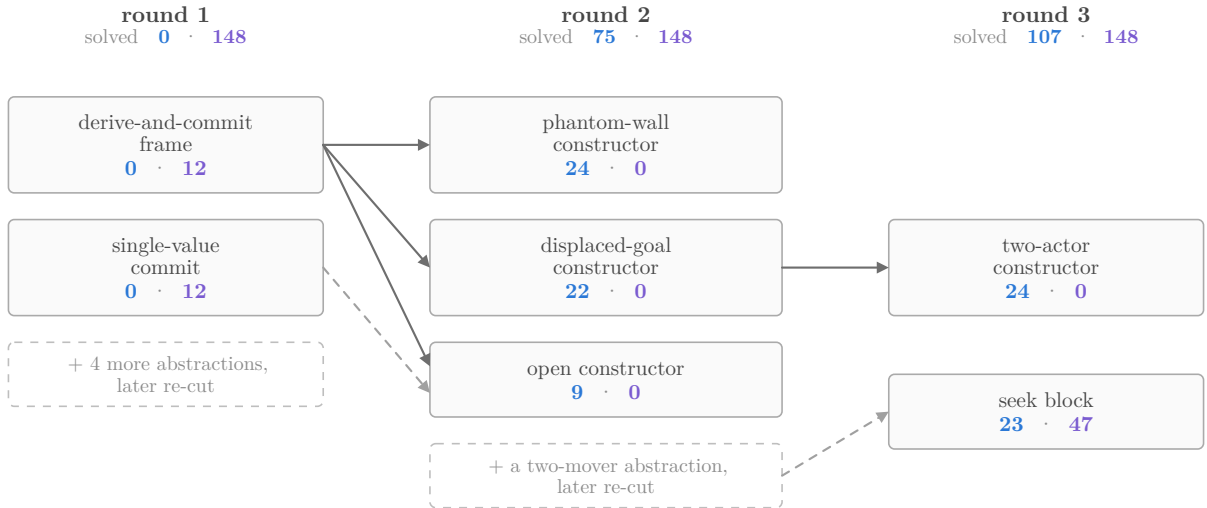
everything except the value the agent is wrong about. In every case the value whose behaviour reaches the world is the value whose reasoning was simulated, and the falsehood stays private. The same agency signature of Section 4.5 manifests in three different ways according to which commit policy is being used.

5.2 The order of arrival

One of the main themes of the developmental literature is the order in which ToM capacities come into fruition Section 1.2. Rational constructivism engages with the timeline evidence most directly, so in our experimental setup we are also interested in the order in which abstractions are acquired.

The figure below denotes each abstraction with a box, in the column of the round during which it was invented. The counts inside the box denote the number of tasks that shared the common structure from which the abstraction was made, and the counts are split by whether it was a solution to a belief task (blue) or non-mental task (violet). The column header has the numbers of the full pool of solutions.

Solid arrows denote that the right-hand abstraction’s body instantiates the left one, that the body of the right-hand abstraction is composed in part of the left-hand abstraction. Solid arrows denote that the single-value commit fills the open constructor’s commit hole; the early wall-and-seek policies are re-cut into the seek block. For the full per-round results, see Table 6.



Round 1 solves 148 tasks, none of which are belief tasks. When the library only has base primitives, every belief family is out of enumeration’s reach. Compression over those 148 non-mental solutions returns six abstractions. Among them are exactly the two target structures of Section 4.5: the *derive-and-commit frame*, of type $(G \rightarrow G) \times (G \times G \rightarrow G) \rightarrow (G \rightarrow G)$, is abstracted from the picture-processing families (overlay, underlay, comet), and the *single-value commit* $V \rightarrow (G \times G \rightarrow G)$, the “agent commit” above (under an earlier symbol) is abstracted from image registration. Neither of them were found from solutions to belief tasks, since no belief tasks were solved yet.

With those two abstractions as single symbols, in the enumeration phase of round 2 the abstractions are composed into more complex belief-attribution programs, which are used to solve 75 belief tasks. The compression phase then invents the phantom-wall, displaced-goal and open

constructors. Round 3 solves 32 more and adds the two-actor constructor and the seek block, re-cutting the early wall-and-seek policies into the latter.

From round 3 the library is fixed. Round 4 only grows uses it more, and the run converges at 132 of 168 belief tasks. An abstraction discovered in round N is a single symbol in round $N + 1$, and the round-1 abstractions are precisely the two capacities (hold a second grid, transfer one named value between grids) that belief needs and that non-mental tasks independently use for. This order just falls out of what is cheap at each stage.

Figure 14 shows how the description length of the corpus decreases as more complex abstractions are introduced. To plot this, we hold the corpus fixed at its base-primitive solutions and re-express every program in each round’s library. The belief families’ total falls from 3,857 to 1,828 nats, and the steepest single drop (1,108 nats) comes at round 2, the round the private-copy constructors are invented. The description length of the non-mental tasks barely moves. Its slight rise after round 1 is the joint compression re-cutting early non-mental spellings toward the structure the belief solutions share.

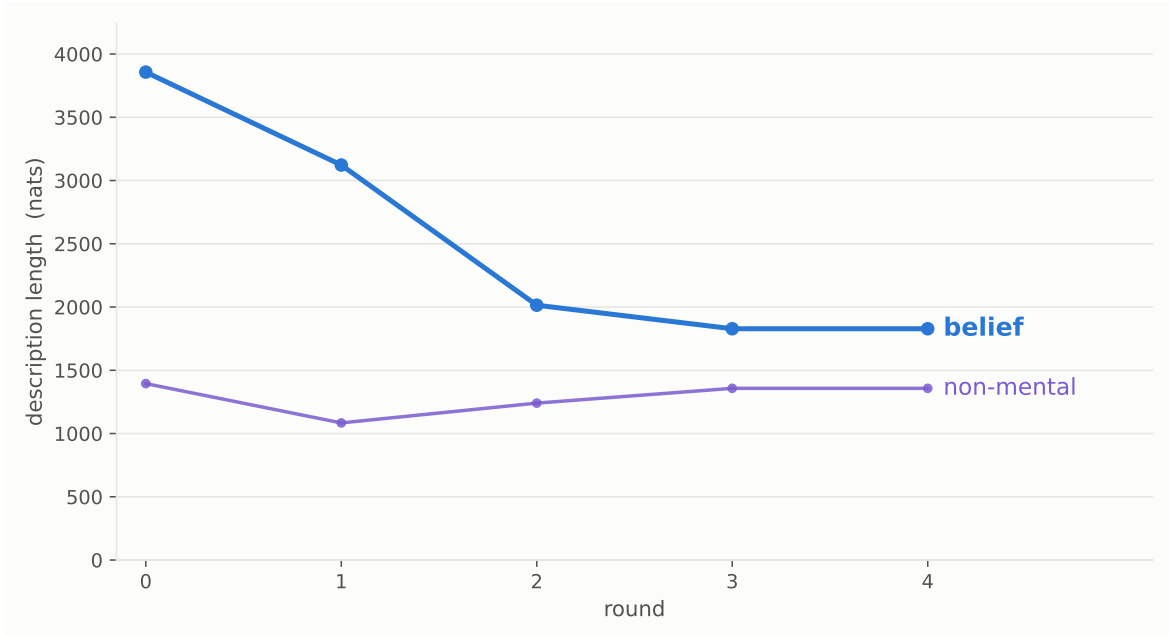


Figure 14: Total description length of the corpus under each round’s library, split into the belief families and the pooled non-mental families.

We also run an “atomic” control run, in the derive-and-commit frame and the single-value commit are included in the initial library as primitives. The atomic control run shows similar results but does everything one round earlier. It solves 55 belief tasks in round 1, and converges in two rounds at 155 of 168. Its final library also has six abstractions, five of which open a private copy and bind the agent value in both the planner and the publication. So it’s the same shape as with the main run, but with coarser granularity, with the agent value shared two ways where the combinator run’s registration spelling shares it three. Its constructors and their bodies are Table 7 and Table 8 in the appendix.

Chapter 6

Limitations and Further Work

6.1 Ablation runs

We currently only have an existence claim that is quite specific to this particular corpus. So in further work we'll regenerate the tasks from the generators at new seeds and then rerun the learner with its final library on them. We'll also run multiple trials over different seeds.

Another experiment we plan to run is to vary the density of the task families in the corpus and measure the round at which the belief attribution abstraction enters the library and the description length it saves (Table 9). The constructivist reading predicts that a concept becomes available when earlier learning has made it cheap, so in this experiment we would test the ratios of each family of tasks and what abstractions they yield.

Another ablation concerns the discount we currently give to visible values. In enumeration our symbols are priced type-uniformly (Section 4.6.1), with the exception that cell values visible in a task's first frame cost zero nats. We're pretty sure this isn't a problem. For example, the census of Table 9 is priced with the exception removed. But the repricing cannot test the exception's effect on search itself. It can't test which programs enumeration reaches within budget, so which solutions compression is shown. So we would re-run the experiment without the exception, to see whether different abstractions are formed.

6.2 Coverage and the budget frontier

Our run doesn't solve all the tasks in the corpus. The run converges at 132 of 168 belief tasks, missing 21 of 48 goal-displacement tasks and 15 of 24 false-obstacle tasks. We argue that these misses are due to budget: the control is exactly as expressive as the combinator run's library, since it just hands the learner two pre-made compositions of primitives. Yet it solves 23 belief tasks that aren't solved in the main run, which indicates that it's lack of budget.

Most solves finish well inside the budget, so the distribution of solved times is as consistent with the misses being out of reach as with their being out of time. Some programs are clearly expressible in our library but are seemingly unreachable by enumeration. Take a scene depicting two agents holding contradictory false beliefs, each detouring around its own phantom wall. That's a scene the language states without difficulty. But it names six latent literals across two nested derive-and-commit frames, and no amount of budget we could give it brought a single such task within reach.

So to settle this we would run a budget dose response experiment. We'd run the same corpus at 600, 1200 and 2400 s per task and beyond. So at some level of budget increase we'd see tasks being solved if the budget was the limiting factor, but any tasks held back by being inexpressible still wouldn't be solved.

6.3 Persistent models, interaction, and observation

The belief our learner attributes is thin, because it's forced that way by the interpreter. A hypothesis is a single-frame transition function $G \times G$. So a program that opens a private channel must build it, use it and collapse it within one step (Section 4.3). Therefore, an attributed model exists only for the duration of one call. Nothing carries it to the next frame, nothing represents how the agent came to hold it, and nothing updates it when the agent looks again. BToM's m' is a belief in the full sense. It's a state the agent came to be in, that could have been otherwise, and that would be revised on new observation (Baker et al., 2009). What our learner finds is just a divergent world model, attributed to an agent and acted on by that agent's policy alone, with no account of acquisition and no ability to revise a belief.

So we would first extend our model by adding the capacity for persistence and interaction. We'd make the interpreter's state the pair rather than the grid, so that an attributed model is threaded from step to step and the program can write to it and read from it across frames. An agent could then genuinely react to its environment. For example it could discover an obstacle by walking into it, record it in its model, and route around it thereafter. As it is now, every divergence is re-posed from scratch at every time step. We actually had this pair-interpreter setup in an earlier version of our model, but we changed to the simpler interpreter.

The second extension we would like to make is to add an observation. Nothing in the present setup represents perceptual access, so the derive just stipulates the divergence, it doesn't explain it. Then scenes with occlusion, and a primitive restricting what an agent's channel receives, would let a false belief be *caused*. The agent believes a wall is there *because* it saw one before it was removed, and stops believing when it sees again.

With persistence and observation together, acquisition and revision become depictable in scenes. And then we can ask whether an MDL learner discovers not just attribution, but updating. Whether it discovers belief as a state with a history, which is a part of BToM but isn't represented in our setup.

6.4 The missing arrow $C \times C \rightarrow U$

No arrow among our primitives carries a coordinate to a utility $C \times C \rightarrow U$. Coordinates are first-class terminals in our domain. The wall stamp primitive and the cell clear primitive consume them, and the reader primitive produces them. But the utility type is inhabited only by the two value-directed distances, so the planner can be aimed at an object but never at a location. Nor does our library offer arithmetic on coordinates from which an offset like "two above" could be formed.

These two absences are what make the extensional reading of the goal-displacement family unstatable with this library, even though "at each step, move toward the cell two above value 2" is the most natural reading of a scene like that. So for that family, the task is solvable only by belief attribution because we didn't include the primitives in the library that would've made it expressible.

Our rationale for excluding $C \times C \rightarrow U$ is that we grant planner as a capacity of the teleological stance, and the teleological stance relates an action to a goal *object*, not to bare space. But of course this rationale doesn't cancel the suspicious effect. At least the false-wall families don't depend on a primitive being omitted, as their extensional rivals are fully expressible in the language (but no

extensional program solves those tasks since they don't generalize across scenes. So the rivals fail for the correct reason).

To discharge this concession, we would go ahead and to the library $\mathcal{L}_0$ a coordinate-valued utility and the offset arithmetic to aim it. If a belief-attribution abstraction still enters the library and still beats the walk-to-a-location program on description length, then the family's verdict is vindicated. If it does not, then the goal-displacement family cannot carry a belief claim and the weight falls entirely on the false-wall families. Until then, the goal-displacement results of Chapter 5 should be read with this condition.

6.5 Metarepresentation

Our learner lacks *metarepresentation*: the grasp of a mental state *as* a representation, something with a referent and a mode of presenting it (Perner, 1991). So our learner can't misrepresent. There is no mode of presentation in a grid. We'd typically use substitution failures to mark an intensional context, but they have no counterpart in one grid's differing from another. So Perner would say that our discovered abstraction is at most at the situation-theoretic stage. It attributes a situation, but doesn't understand a representation as a representation. Our central claim survives this challenge, because it rests only on attribution. The divergent model is indeed *somebody's*, and only that somebody's movement is determined based on the model.

But doing more than that is hard, because grids are extensional objects. But the setup already contains the ingredient a mode of presentation needs, namely a representation with syntax. The derive is a program, and the attributed model is only its output. A learner that attributed the *derive* itself rather than the grid would have the beginnings of an intensional context. Substituting one for the other could change the agent's behaviour in scenes where the derives come apart. But even if the programs compose and give use the attribution of a derive like that, it would be fiendishly hard to design task families that force this distinction.

6.6 Recursive mindreading

The two-observer family puts two agents in one scene and asks only whether the learner attributes a private model to the right one. What it never asks is how the two agents stand to *each other*. Nothing in the corpus depicts one agent modelling another, and nothing in the discovered abstraction could express it if it did: the constructor opens a private copy of the *grid*, and a grid has no room in it for somebody else's model.

The natural target is the group case. Shum et al. extend inverse planning to scenes where behaviour is generated by relations — who is cooperating with whom, who is working against whom — and represent those relations as compositions of a small set of team operators, applied recursively, so that arbitrary team structures are built out of a compact set of building blocks (Shum et al., 2019). That is the representation this thesis's question should be put to next, and putting it there is the same move made one level up: their operators are given, as `fork` and `sync_to_world` were given in the atomic control, and the question is whether a learner shown enough scenes of agents helping and hindering each other would assemble them out of the plumbing it already has, or whether they have to be installed.

Two things would have to change first. The corpus would need scenes whose generating program cannot be written as two independent agent blocks side by side — one agent’s policy conditioned on where another agent is going, which the present single-frame transition functions cannot state (Section 6.3) — and the type system would need channel arity as a free parameter, a recursive stack of grids in place of the fixed pair, since an agent modelling an agent is a private channel nested inside a private channel and the pair $G \times G$ permits exactly one. The prediction, if the thesis’s argument is right, is that the depth of nesting selected by compression should track the depth the scenes actually require, and stop there.

Chapter 7

Conclusion

In this thesis we studied the acquisition of Theory of Mind. In Chapter 1 we surveyed the developmental literature, where we identified belief attribution as a core capacity of ToM which is much studied experimentally. We then compared theoretical perspectives of ToM acquisition and argued that rational constructivism, which models ToM development as theory change, is the most compelling among the accounts.

Rational constructivism posits that domain-specific conceptual vocabulary is built by domain-general inference over structured hypotheses, guided by a preference for simpler hypotheses (Section 1.4.4). We identified that the nativist’s strongest reply to the constructivist is to deny that mechanisms for learning structured domains can exist. So we set out to answer this charge by counterexample, via a model that can learn the structure of ToM.

In Chapter 2 we explained the learning framework that we chose to base our model in. We justified our choice of representing hypotheses as programs, and showed how we can use Bayesian inference to synthesize programs, by taking the prior over them is the measure of their simplicity. We showed that similar inference methods can be used to learn the library itself, compressing recurring structure into new abstractions and adding them to the library.

In Chapter 3 we surveyed BToM as the best available model of the mature capacity for ToM. We showed how an observer uses inverse planning to infer an agent’s belief, inferring under which falsely believed model an agent’s irrational behavior would be rational. We then identified the structure that a BToM modeler ordinarily stipulates (Section 3.4), so that we could ensure that none of those features are smuggled into our own setup.

In Chapter 4 we introduced our model of a learner that isn’t endowed with the stipulations given in BToM. We created a corpus of grid-based tasks among which some require belief attribution. We gave our learner a primitive library of a planner, a symmetric pair type, and general-purpose functions over pairs. We showed how the learner could in principle compose its domain-general primitives into programs that attribute beliefs. And we spelled out the criteria under which our learner would be evaluated.

In Chapter 5 we showed the results our implementation of the model yielded. Our learner’s library of discovered abstractions satisfied most of the criteria. The abstractions open a private model of the world, make the model false, run an agent’s planner against the model rather than the world, and publish back the agent’s resulting movement.

The commit is spelled three different ways across the abstractions, yet in every one of them the value whose behavior reaches the world is the value whose reasoning was simulated. That

correct binding makes the abstractions *attributions* and not just counterfactuals. The falsehood is *somebody's*.

The library's order of arrival is also evidence in and of itself. No belief tasks are solved in round one, then compression over the solutions to those non-mental tasks yields precisely the two capacities that belief needs: holding a second grid, and transferring a value between grids. Once those are added as new abstractions, the belief families are solved in rounds two and three.

The atomic control run confirms the reading from the other side. Handed the same two capacities outright as primitives, the learner is able to solve those same tasks a round earlier. This is in keeping with the theory's account of the developmental timeline (Section 1.3): belief attribution arrives late because it is a composition whose components must themselves first be assembled.

Our result is an existence claim. There exists a learning process that constructs terms encoding the structure of belief attribution, where the learner begins only with domain-general primitives. But our demonstration is quite limited: it covers one corpus and one initial library. The belief attributed is a thin, single-step divergence with no acquisition or revision, no metarepresentation, and no recursive mind reading. And of course, we make no claim that children actually learn ToM this way. Our claim is at the computational level.

Within its scope, though, the result is hopefully enough to keep the nativist at bay. Our learner was shown behavior and was equipped only to prefer short hypotheses. It added the belief attribution abstraction to its library because naming the structure "an agent acting on its own model of the world" made the observed behavior shorter to describe than descriptions told in terms of the world alone. If our learner is anything like a child, then Theory of Mind really is a *theory*. Its unobservables posited because the world is more compactly explained with them than without.

Chapter 8.

Appendix

Reference material for Chapters 4 and 5: the corpus, the primitives handed to the learner, the libraries it converged to, and the per-family cost census.

8.1. The task corpus

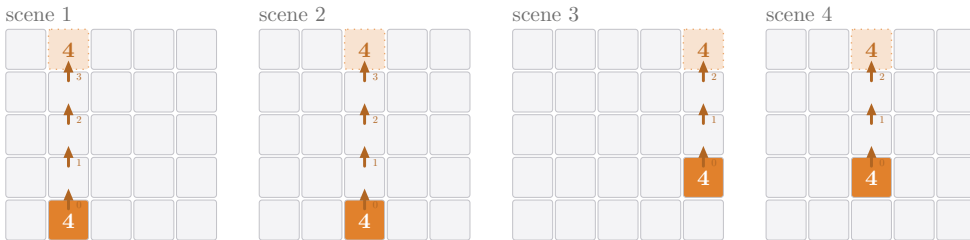
The 168 belief tasks are 24 false-wall, 48 goal-displacement, 48 witness, 24 two-observer and 24 false-obstacle; the remaining 148 are spread across the twenty non-mental families. Every task consists of $k = 4$ scenes on 5×5 grids, and the (goal, agent) values rotate through eight distinct pairs, so that no particular integer comes to mean “agent”. The non-mental side is deliberately not uniform: obstacle is the densest family at 48 tasks and relocation carries 16, because the wall-handling structure they share with belief must recur often enough for compression to abstract it — a deliberate curriculum choice.

One task of each family, all four of its scenes drawn side by side, each as the trajectory collapsed onto its opening grid: one numbered arrow per transition, cells reached only later ghosted in and deletions marked. The scenes of a task share one latent program, and a solution counts only if it reproduces every one of them.

empty
 1 id 1
 2 id 2
 3 wall
 4 id 4
 5 id 5
 posited by the program

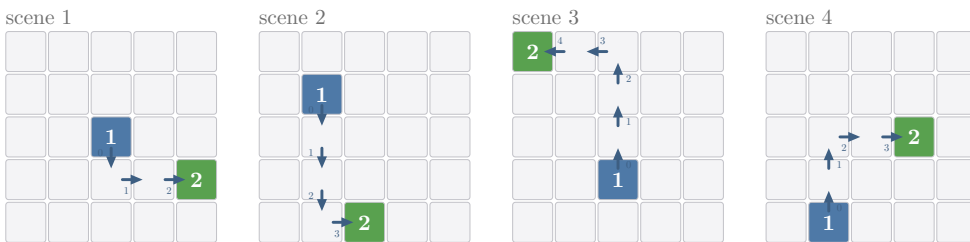
Constant movement [physics]

Value 4 drifts one cell up per frame; nothing else on the grid changes.



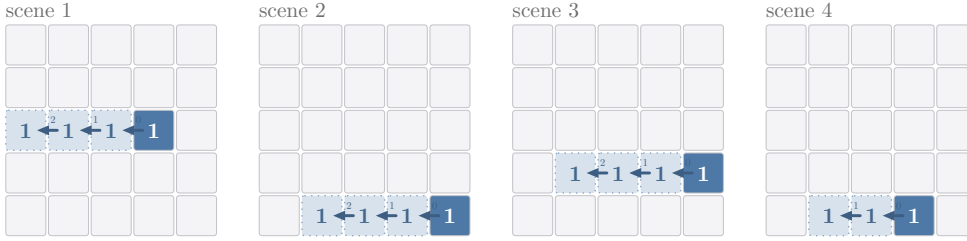
Goal-directed movement [desire]

Value 1 wants value 2 and takes the shortest route to it, one step per frame. It sees the grid as it is, so nothing separates what it takes to be the case from what is.



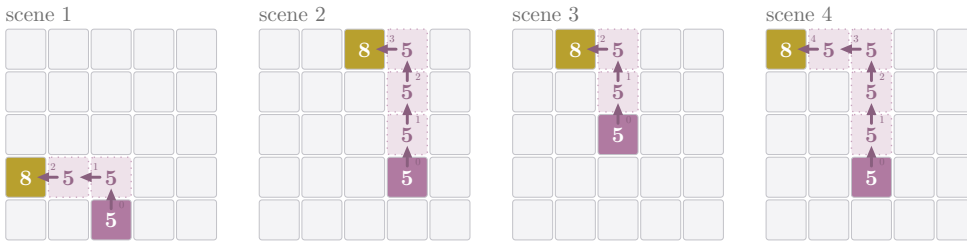
Overlay ^[overlay]

Value 1 steps left, but the cells it leaves are never cleared, so it smears across the grid rather than moving through it.



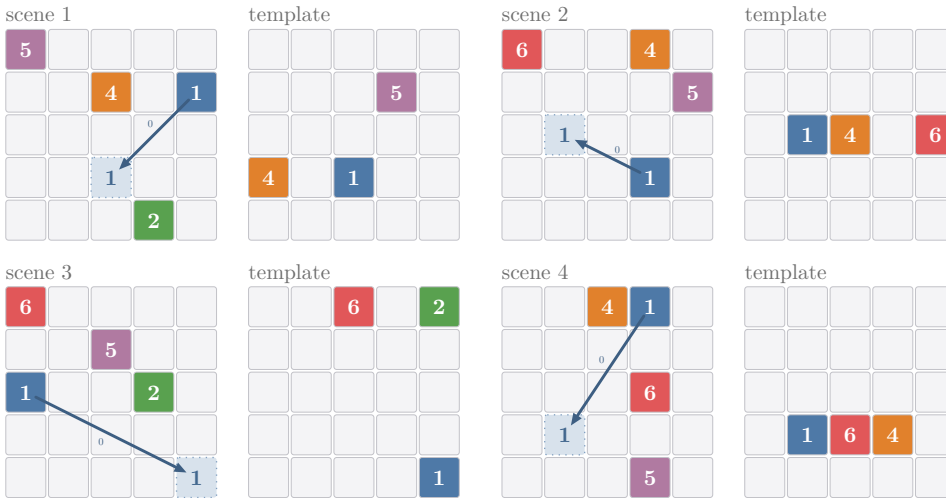
Comet ^[comet]

Value 5 walks to value 8 as in goal-directed movement, but keeps every cell it has passed through: goal-directed motion and a trail, with no mind in it.



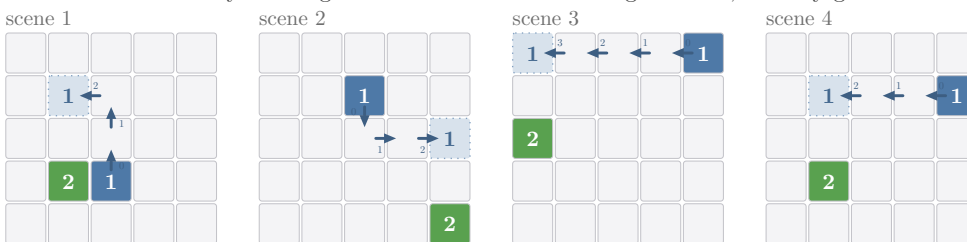
Registration ^[registration]

Value 1 is moved to the cell the template assigns it. Every other value on the canvas stays where it stands, including the ones the template disagrees about.



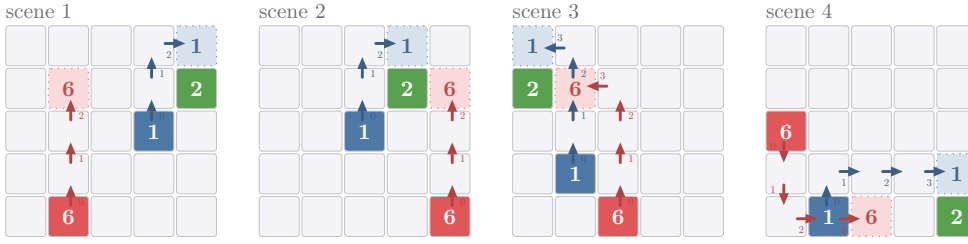
Goal displacement ^[belief_goal]

Value 2 never moves. Value 1 walks to (1, 1) and stops there, because in its own model the goal has been shifted away from where it actually lies: it goes to where it believes the goal to be, as Sally goes to her own basket.



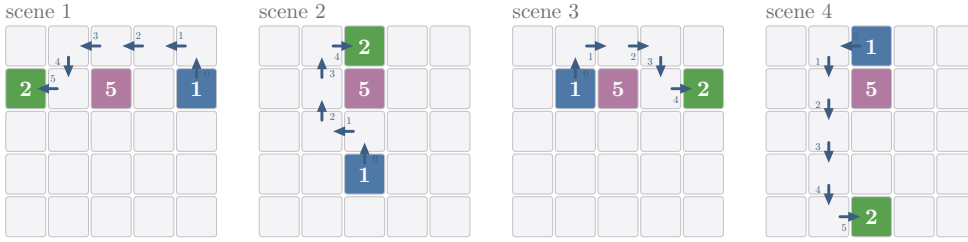
Two observers [belief_observers]

Value 2 stays where it is. Value 1 heads for (0, 4), the cell where it believes the goal to be, while value 6 heads for the goal itself. One scene, a false belief attributed to the first agent and none to the second.



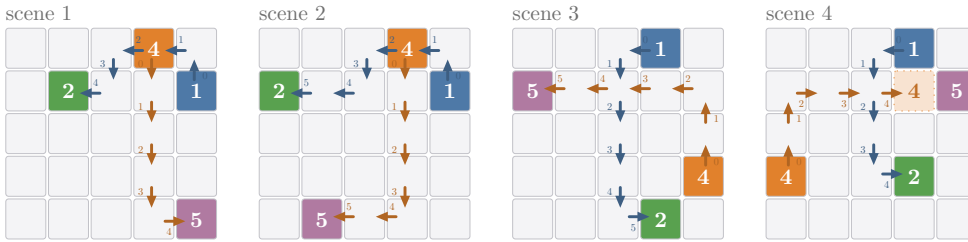
False belief [belief_wall]

Value 1 heads for value 2 but believes a wall stands at (1, 2), so it bends round that cell and pays extra steps for it. No frame of the task contains a wall, and value 5 is standing on the cell in question throughout, untouched.



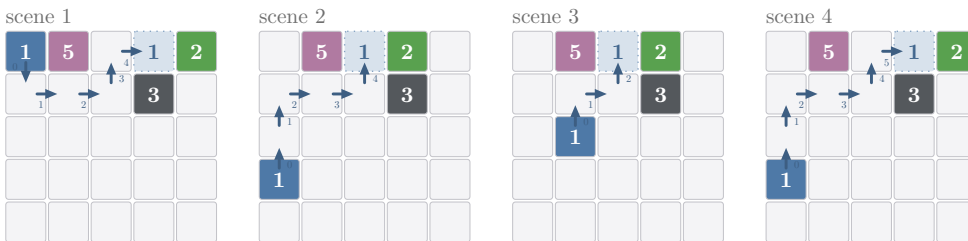
Witness belief [belief_witness]

Value 1 believes a wall stands at (1, 3) and detours round it on its way to value 2. Value 4, heading for value 5, walks straight through that same cell. The wall has to be attributed to one agent and withheld from the other.



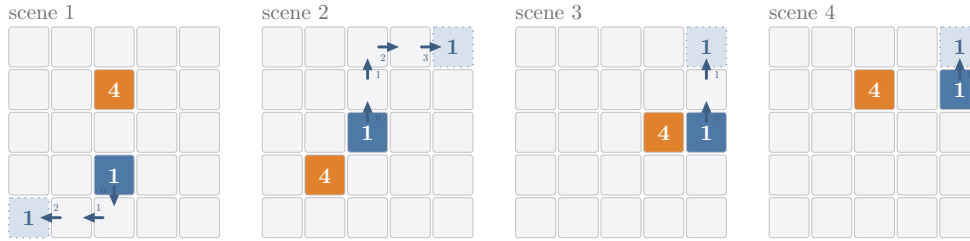
False obstacle [belief_false_obstacle]

Value 1 is mistaken twice over: it believes a wall stands at (0, 1), where value 5 is in fact standing, and that value 2 lies at (0, 3). So it bends round an empty cell and settles one short of the goal. The wall at (1, 3) is the real one, and stays on the grid throughout.



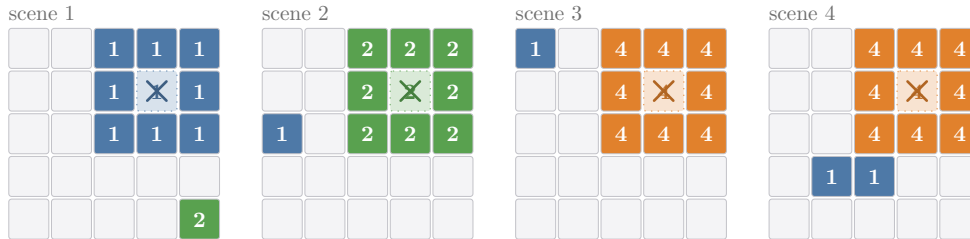
Flee ^[flee]

Value 1 moves to whichever neighbouring cell is furthest from value 4, and stops once no move gains it anything.



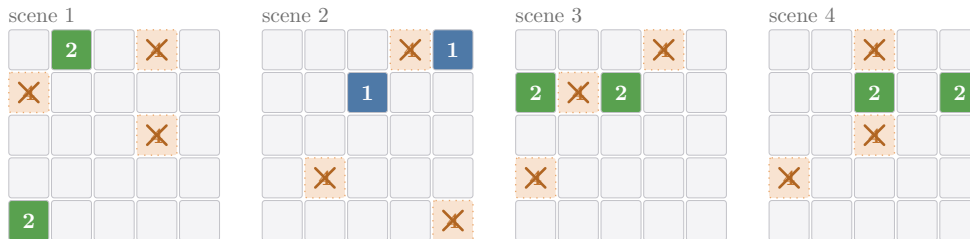
Deletion ^[deletion]

Cell (1, 3) is cleared, whatever was standing on it; the rest of the grid is untouched.



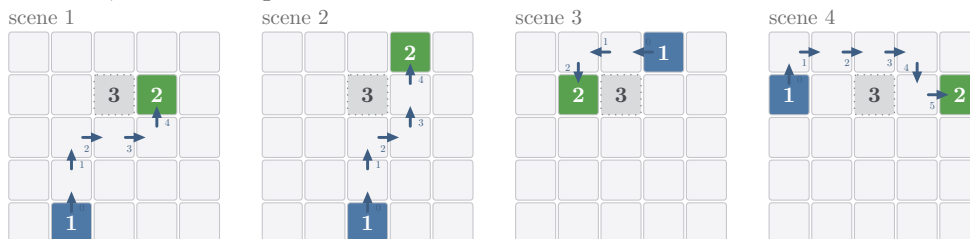
Denoise ^[denoise]

Every cell holding value 4 is erased at once, wherever it lies; everything else is kept.



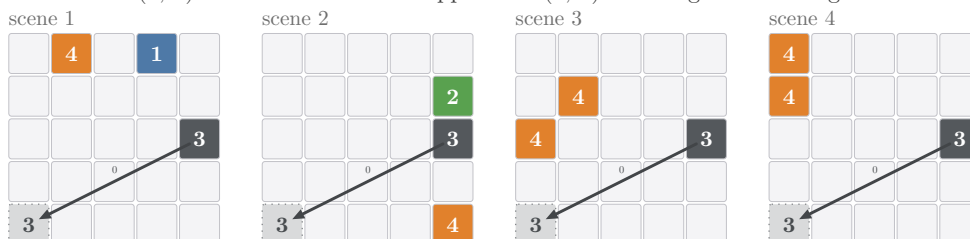
Obstacle ^[obstacle]

A wall is put on the grid at (1, 2), and value 1 walks round it to reach value 2. The detour has the shape of the false-belief one, but here the grid itself shows what causes it.



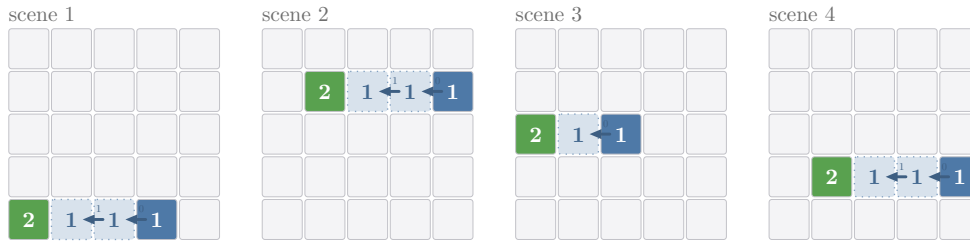
Relocation ^[relocate]

The wall at (2, 4) is cleared and a wall appears at (4, 0). Nothing else on the grid moves.



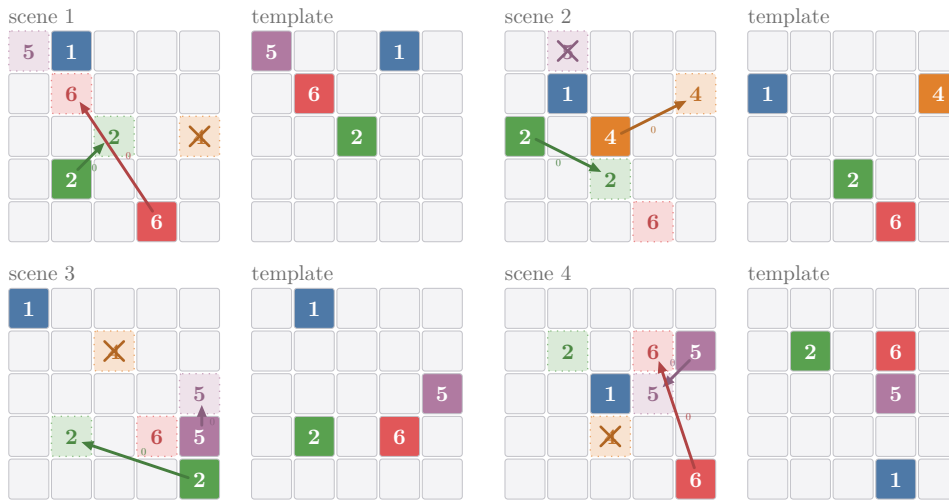
Underlay [underlay]

Value 1 steps left and leaves a trail, but wherever the grid is already occupied the standing value wins: the trail stops at value 2 instead of painting over it.



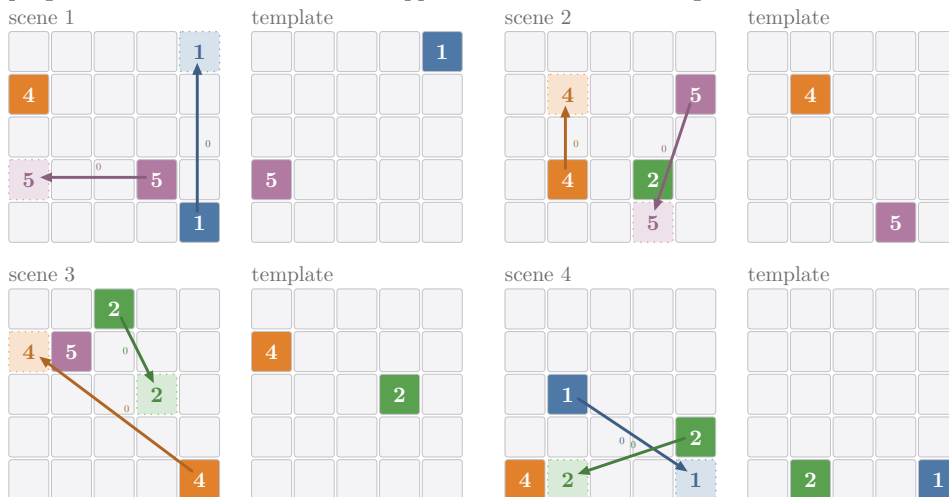
Perception [perception]

The template is updated to say where value 1 actually is on the canvas; everything else the template holds is left as it was. The world is read but not changed.



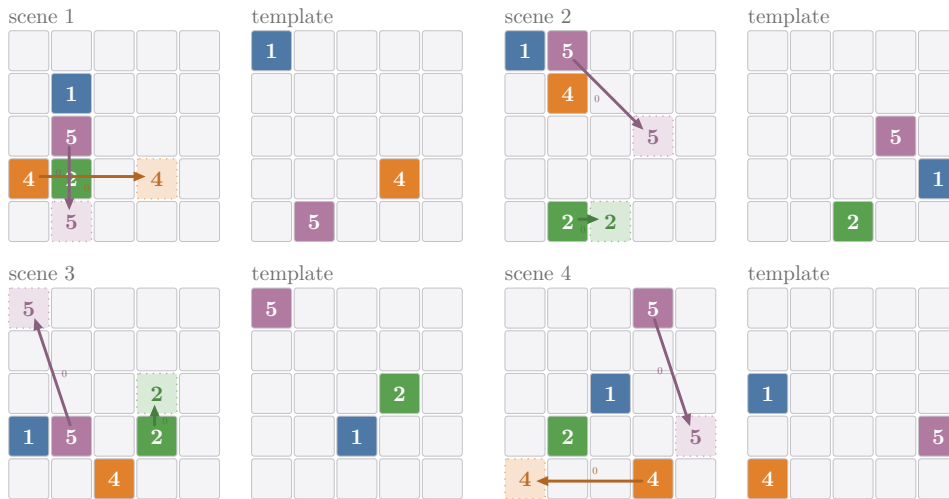
Multi-registration [multi_reg]

Every value the template also carries is moved to the cell the template assigns it; values only the canvas has stay put. The program names no value at all — it applies to whatever the two grids share.



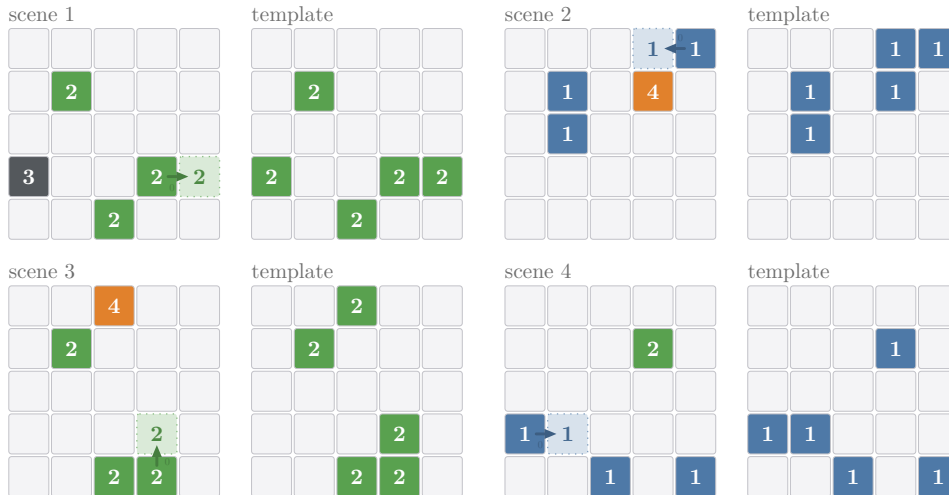
Registration-except [reg_except]

Every value is snapped to its template cell except value 1, which keeps the position it has on the canvas: one object exempted from a rule the rest are subject to.



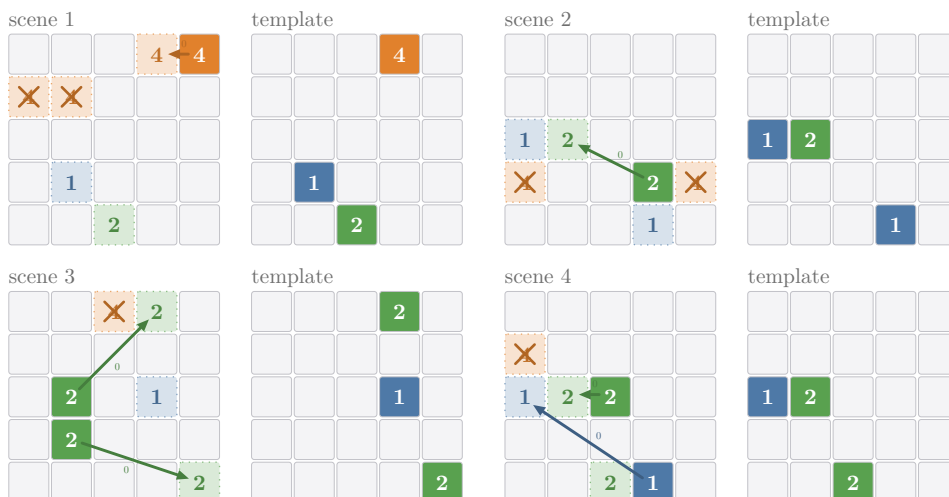
Inpainting [inpaint]

Wherever the canvas is empty the template's value is filled in; wherever the canvas already holds something, the canvas wins.



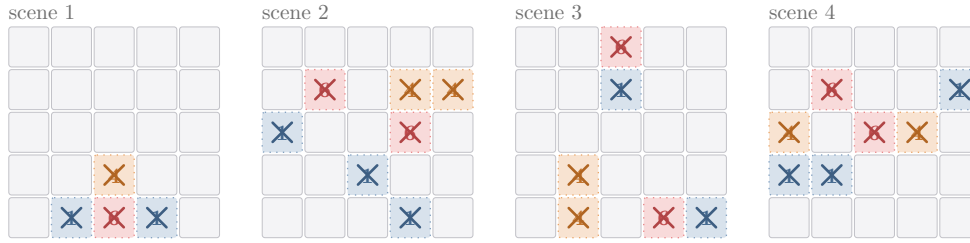
Readout [readout]

The template is returned as it stands and the canvas is thrown away: the answer is whatever the second grid says, however the first one looks.



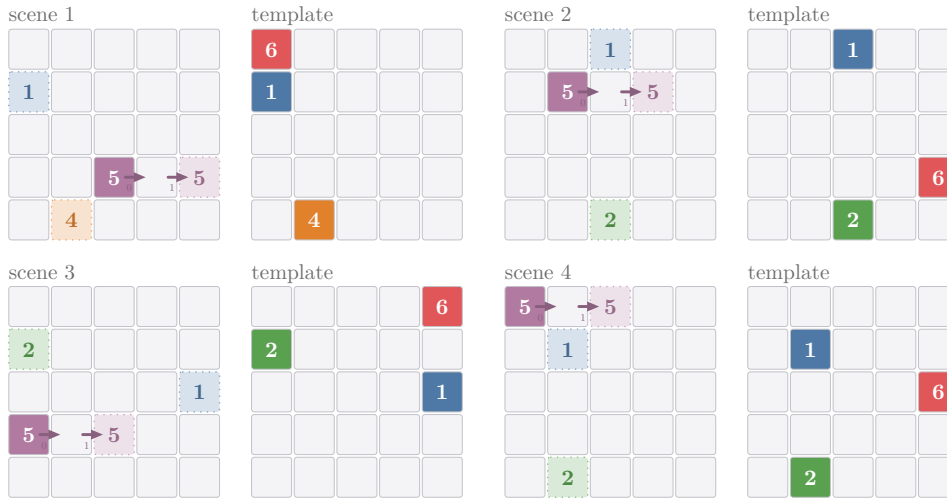
Wipe [wipe]

Every value on the grid is cleared in a single step, and the blank grid then stays blank.



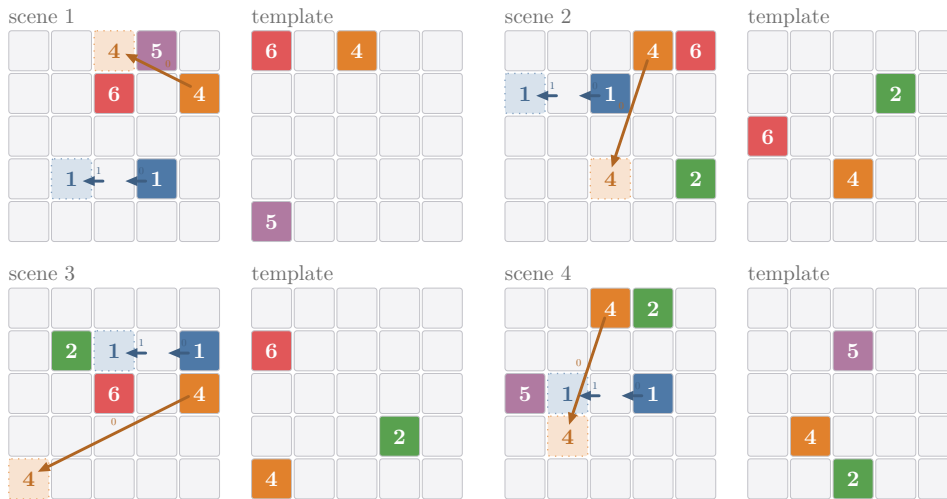
Composite [composite]

Value 5 steps right on one layer while value 6 is erased from the other, and the two layers are then flattened into a single grid.



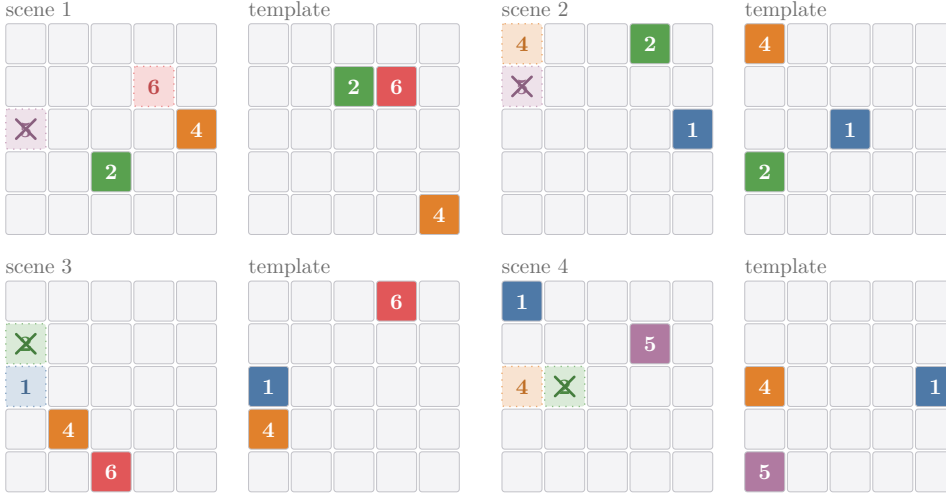
Drifting registration [drift_reg]

Value 4 is moved to the cell the template assigns it while value 1 steps left at the same time: a registration carried out inside a moving image.



Map update [map_update]

The two grids trade roles and everything is then snapped: the template takes on the canvas's position for every value they both hold, keeps value 6, which only it has, and does not take up value 5, which only the canvas has.



8.2. The primitive catalogue

The primitive sets handed to the learner, summarised in Section 4.4: every symbol's repr, type signature and semantics, under each endowment.

Symbol	Type	Semantics
compose	$(G \rightarrow G)^2 \rightarrow (G \rightarrow G)$	run one transition then the other
step	$V \times D \rightarrow (G \rightarrow G)$	move every cell of the given value one step in the direction
optimize	$U \times V \rightarrow (G \rightarrow G)$	move the given value one greedy (BFS-optimal) step to improve the utility
neg_dist	$V \rightarrow U$	minus the distance to the nearest cell of the target value (attract)
distance	$V \rightarrow U$	plus that distance, so maximising it flees the target
wall_at	$C \times C \rightarrow (G \rightarrow G)$	stamp a wall at a position
clear_at	$C \times C \rightarrow (G \rightarrow G)$	clear one cell
erase	$V \rightarrow (G \rightarrow G)$	remove every cell of a value
right left up down	D	the four movement directions
c0 ... c4	C	coordinate terminals, a pair naming a position (the invisible latent pool)
0 ... 9	V	cell-value terminals (content-priced when visible)

Table 2: The grid-edit primitives that move cell values about a single grid

Symbol	Type	Semantics
dup	$G \rightarrow G \times G$	pair a grid with a copy of itself
mapsnd	$(G \rightarrow G) \rightarrow (G \times G \rightarrow G \times G)$	$(w, m) \mapsto (w, f(m))$: act on the second channel, leave the first
compose_gp	$(G \rightarrow G \times G) \times (G \times G \rightarrow G \times G) \rightarrow (G \rightarrow G \times G)$	build a pair, then transform it
pipe_gpg	$(G \rightarrow G \times G) \times (G \times G \rightarrow G) \rightarrow (G \rightarrow G)$	produce a pair, then consume it
locate	$V \rightarrow (G \rightarrow C \times C)$	where a named value is in a grid
place	$V \rightarrow (G \times (C \times C) \rightarrow G)$	move a named value to a position
register	$(G \rightarrow C \times C) \times (G \times (C \times C) \rightarrow G) \rightarrow (G \times G \rightarrow G)$	$(w, m) \mapsto \text{plc}(w, \text{loc}(m))$: read a position off one channel, impose it on the other
overlay	$G \times G \rightarrow G$	union the channels, the second wins ties — a <i>non-mental</i> commit
underlay	$G \times G \rightarrow G$	union the channels, the first wins ties
sync_all	$G \times G \rightarrow G$	move <i>every</i> shared value to its position in the second channel
sync_except	$V \rightarrow (G \times G \rightarrow G)$	every shared value but one
fst / snd	$G \times G \rightarrow G$	project the first channel / the second
swap	$G \times G \rightarrow G \times G$	exchange the channels
mapfst	$(G \rightarrow G) \rightarrow (G \times G \rightarrow G \times G)$	$(w, m) \mapsto (f(w), m)$: act on the <i>first</i> channel
bimap	$(G \rightarrow G)^2 \rightarrow (G \times G \rightarrow G \times G)$	$(w, m) \mapsto (f(w), g(m))$: act on both, one function each
pair_blank	$G \rightarrow G \times G$	pair a grid with a fresh blank rather than a copy
compose_pg	$(G \times G \rightarrow G \times G) \times (G \times G \rightarrow G) \rightarrow (G \times G \rightarrow G)$	transform a pair, then consume it — the third composition, and the only one that acts on a pair the program did not build

Table 3: The pair combinators

Symbol	Type	Semantics
fork	$(G \rightarrow G) \times (G \times G \rightarrow G) \rightarrow (G \rightarrow G)$	the derive-and-commit frame of Section 4.5, bought whole: copy the grid, run the derive on the copy, hand the (world, copy) pair to the commit
sync_to_world	$V \rightarrow (G \times G \rightarrow G)$	the single-value commit bought whole: move v in the world to its position in the model, return the world
sync_to_model	$V \rightarrow (G \times G \rightarrow G)$	the direction complement, also atomic here: move v in the model to its position in the world, return the model — perception rather than action
then_sync	$(G \times G \rightarrow G) \times V \rightarrow (G \times G \rightarrow G)$	run the given commit, then <code>sync_to_world</code> v on its result. Absent from the combinator endowment, where it would smuggle an atomic <code>sync_to_world</code> back in — with it, registration solves as <code>(then_sync fst_gg v)</code> and never reaches for the register/locate/place spelling
sync_all / sync_except	$G \times G \rightarrow G$ / $V \rightarrow (G \times G \rightarrow G)$	the scope collapses, atomic in both endowments (Table 3)
overlay / underlay	$G \times G \rightarrow G$	the z-order unions, atomic in both endowments
fst_gg / snd_gg	$G \times G \rightarrow G$	the projections — the <code>fst/snd</code> of Section 4.4, under the run’s reprs — atomic in both endowments

Table 4: The atomic control’s pair interface, which hands the learner pre-composed higher-level primitives

8.3. The final libraries

The final library of the combinator run. The run’s names, the types, the argument orders, and the bodies are expanded to base primitives. The arguments an abstraction takes are written `$0`, `$1`, ... in the body and are supplied in that order, which is the run’s own and carries no meaning.

Symbol	Type, in argument order	Body
fn_6	$V \times V \times (G \rightarrow G) \times (G \times G \rightarrow G) \rightarrow (G \rightarrow G)$	(pipe_gpg (compose_gp dup (mapsnd (compose \$2 (optimize (neg_dist \$1) \$0)))) \$3)
fn_7	$V \times D \times V \times V \rightarrow (G \rightarrow G)$	(compose (optimize (neg_dist \$0) \$3) (pipe_gpg (compose_gp dup (mapsnd (compose (step \$0 \$1) (optimize (neg_dist \$0) \$2)))) (sync_except \$0))))
fn_8	$V \times V \times (G \rightarrow G) \rightarrow (G \rightarrow G)$	(compose \$2 (optimize (neg_dist \$1) \$0))
fn_9	$C \times C \times V \times V \rightarrow (G \rightarrow G)$	(pipe_gpg (compose_gp dup (mapsnd (compose (wall_at \$1 \$0) (optimize (neg_dist \$2) \$3)))) sync_all)
fn_10	$V \times D \times V \rightarrow (G \rightarrow G)$	(pipe_gpg (compose_gp dup (mapsnd (compose (step \$0 \$1) (optimize (neg_dist \$0) \$2)))) (sync_except \$0))
fn_11	$V \rightarrow (G \times G \rightarrow G)$	(register (locate \$0) (place \$0))

Table 5: The final library

The library shown in Table 5 lists the abstractions found in round 3 on, since no new abstractions are found in round 4; the library is just reused to solve new tasks.

Round	Symbol	Type, in argument order	Body
1	fn_0	$V \times V \times C \times C \rightarrow (G \rightarrow G)$	(compose (wall_at \$3 \$2) (optimize (neg_dist \$1) \$0))
1	fn_1	$V \rightarrow (G \times G \rightarrow G)$	(register (locate \$0) (place \$0))
1	fn_2	$(G \rightarrow G) \times (G \times G \rightarrow G) \rightarrow (G \rightarrow G)$	(pipe_gpg (compose_gp dup (mapsnd \$0)) \$1)
1	fn_3	$C \times C \times C \times C \rightarrow (G \rightarrow G)$	(compose (wall_at \$3 \$2) (clear_at \$1 \$0))
1	fn_4	$V \times V \rightarrow (G \rightarrow G)$	(optimize (neg_dist \$0) \$1)
1	fn_5	$V \times V \times C \rightarrow (G \rightarrow G)$	(compose (wall_at \$2 c2) (optimize (neg_dist \$0) \$1))
2	fn_6	$V \times V \times (G \rightarrow G) \times (G \times G \rightarrow G) \rightarrow (G \rightarrow G)$	(pipe_gpg (compose_gp dup (mapsnd (compose \$2 (optimize (neg_dist \$1) \$0)))) \$3)
2	fn_7	$V \times V \times C \times C \rightarrow (G \rightarrow G)$	(compose (wall_at \$3 \$2) (optimize (neg_dist \$1) \$0))
2	fn_8	$V \times V \times C \times V \times V \rightarrow (G \rightarrow G)$	(compose (pipe_gpg (compose_gp dup (mapsnd (compose (wall_at \$2 c2) (optimize (neg_dist \$3) \$4)))) sync_all) (optimize (neg_dist \$1) \$0))
2	fn_9	$V \times D \times V \rightarrow (G \rightarrow G)$	(pipe_gpg (compose_gp dup (mapsnd (compose (step \$0 \$1) (optimize (neg_dist \$0) \$2)))) (sync_except \$0))
2	fn_10	$V \rightarrow (G \times G \rightarrow G)$	(register (locate \$0) (place \$0))
2	fn_11	$C \times C \times V \times V \rightarrow (G \rightarrow G)$	(pipe_gpg (compose_gp dup (mapsnd (compose (wall_at \$1 \$0) (optimize (neg_dist \$2) \$3)))) sync_all)

Table 6: Rounds 1 and 2, with the bodies expanded to base primitives.

Table 6 shows the abstractions found at the end of round 1 and 2. Round 1’s fn_2 is the derive-and-commit frame and its fn_1 the single-value commit, both discovered before any belief task is solved; round 2’s fn_6, fn_8, fn_9 and fn_11 are the first four abstractions that open a private copy, and fn_8 is the two-mover abstraction round 3 discards. Note the literal c2 where the final library’s two-actor constructor carries a hole. The compression step re-derives the library from scratch each round, so symbols are not stable across rounds; fn_7 below and fn_7 above are different abstractions.

Section 5.1 also summarizes the control run. Its five constructors differ in what is done to the private copy. Here is what each does at a time step, with its arguments as free letters, and the bodies they are read from below.

Symbol	What it does at each time step
<code>fn_6(a, g, δ)</code>	on a private copy of the grid altered in some unspecified way δ , a moves to the adjacent cell closest to g ; then a 's new position, and nothing else, is written back to the world
<code>fn_7(a, g, δ)</code>	the alteration δ is made to the grid itself, and then a moves to the adjacent cell closest to g — no copy is opened
<code>fn_8(d, b, a, n)</code>	n moves to the adjacent cell closest to b ; then, on a private copy in which b has been displaced one cell in direction d , a moves to the adjacent cell closest to b ; then a 's position alone is written back
<code>fn_9(u, v)</code>	on a private copy: u moves to the adjacent cell closest to v and then to the adjacent cell closest to the nearest 0, and u 's position is written into that copy; still in the copy, v then moves to the adjacent cell closest to u ; finally v 's position alone is written back to the world
<code>fn_10(c, g, a, g', n)</code>	on a private copy with an impassable wall stamped in row 2 at column c , a moves to the adjacent cell closest to g , and a 's position alone is written back; then, in the world, n moves to the adjacent cell closest to g'
<code>fn_11(c, r, g, a)</code>	on a private copy with an impassable wall stamped at row r , column c , a moves to the adjacent cell closest to g ; then a 's position alone is written back

Table 7: The final abstractions from the atomic control, read as prose.

One remark: what `fn_9` does to its private copy itself opens a second private copy, so the discovered shape embeds — though the inner copy is part of the outer agent's model rather than a model of another mind, so this is not second-order mentalizing in the sense of Section 1.2.

Symbol	Body	Belief solves
<code>fn_6</code>	<code>(fork (compose \$2 (optimize (neg_dist \$1) \$0)) (sync_to_world \$0))</code>	37
<code>fn_7</code>	<code>(compose \$2 (optimize (neg_dist \$1) \$0))</code>	18
<code>fn_8</code>	<code>(compose (optimize (neg_dist \$1) \$3) (fork (compose (step \$1 \$0) (optimize (neg_dist \$1) \$2)) (sync_to_world \$2)))</code>	24
<code>fn_9</code>	<code>(fork (compose (fork (compose (optimize (neg_dist \$1) \$0) (optimize (neg_dist 0) \$0)) (sync_to_world \$0)) (optimize (neg_dist \$0) \$1)) (sync_to_world \$1))</code>	14
<code>fn_10</code>	<code>(compose (fork (compose (wall_at c2 \$0) (optimize (neg_dist \$1) \$2)) (sync_to_world \$2)) (optimize (neg_dist \$3) \$4))</code>	29
<code>fn_11</code>	<code>(fork (compose (wall_at \$1 \$0) (optimize (neg_dist \$2) \$3)) (sync_to_world \$3))</code>	52

Table 8: The final library under the atomic control, bodies expanded to base primitives

Five of the six carry the agency signature; `fn_7` is the bare seek policy (move a toward g) with room for one edit in front of it, which belief shares with the obstacle family (47 obstacle solves are rewritten through it).

8.4. The per-family description-length census

The census below prices every family’s solutions before and after the library was learned.

Family	Solves priced	Base (nats)	Final library	Saved
belief	132	34.51	10.78	17.11
obstacle	48	15.97	12.79	3.18
registration	4	6.80	4.50	2.30
perception	4	10.39	8.08	2.30
drift_reg	4	16.56	14.26	2.30
goal-directed movement	16	7.78	7.78	0.00
constant movement	4	6.17	6.17	0.00
comet	4	16.05	16.05	0.00
overlay / underlay	4 each	14.44	14.44	0.00
composite	4	16.74	16.74	0.00
relocate	16	13.89	13.89	0.00
flee	4	7.78	7.78	0.00
deletion	4	5.70	5.70	0.00
denoise	4	4.79	4.79	0.00
wipe / map_update	4 each	5.78	5.78	0.00
multi_reg / reg_except / inpaint / readout	4 each	2.20–4.50	unchanged	0.00

Table 9: Median description length per task, before and after the learned library, over all 280 solved tasks.

The census is priced under the type-uniform prior alone, that is, with Section 4.6.1’s visible-value exception removed. Repricing the same programs with the exception restored at half and at full strength (the prior enumeration actually searches under) leaves belief’s median saving at 17.11 nats in all three cases, while the three registration-flavoured savings fall from 2.30 to 1.15 to exactly 0.00: their whole economy is naming a visible value once instead of twice, and once the discount makes visible values free there is nothing left to save. Under the full discount, belief and obstacle are the only families the library shortens at all. Within belief, the wall and witness variants are invariant to the last digit; the goal-displacement, two-observer and false-obstacle variants each give back 2.30 nats — the uniform cost of one cell-value terminal — per visible value their abstraction absorbs (two, three and one respectively), and still save between 9.25 and 19.59 nats with the discount at full strength.

8.5. The interpreters

Iterating a transition function from an initial frame is structure common to every solution, so it is kept out of program space and put in the evaluation framework instead (Section 4.3). Were it in program space it would be baked into every abstraction the learner discovers — a corpus of falling-object scenes would yield not the reusable (**step down \$0**) but an overspecified “apply (**step down \$0**) from x_0 for \$1 steps”, whose output type is an entire rendered scene and which therefore

composes with nothing. Keeping the recurrence out of program space keeps every abstraction an arrow $G \rightarrow G$.

A trajectory task is checked with the standard unfold: its state is a single grid, not a (world, model) pair, and not a registry of agents or goals.

```
def unfold(g: grid, T: int, f: fn) -> mat:
    frames = [g.copy()]
    for _ in range(T - 1):
        g = f(g)
        frames.append(g.copy())
    return np.stack(frames)
```

A template task starts from a pair, so it is checked with a second interpreter, which threads the canvas and re-pairs each frame with the same constant external template.

```
def unfold_with_template(g, template, T, c):
    """grid, grid, int, fn_p_g -> mat: thread g; each frame pair it with a
    *constant external* template and apply commit c. Unlike fork, the second
    channel is a given input, not a privately derived model – so a program that
    uses sync_to_world here is doing registration, not holding a belief.
    """
    frames = [g.copy()]
    for _ in range(T - 1):
        g = c((g, template))
        frames.append(g.copy())
    return np.stack(frames)
```

Bibliography

- Apperly, I. A., & Butterfill, S. A. (2009). Do humans have two systems to track beliefs and belief-like states?. *Psychological Review*, 116(4), 953–970. <https://doi.org/10.1037/a0016923>
- Baker, C. L. (2012). *Bayesian Theory of Mind: Modeling Human Reasoning about Beliefs, Desires, Goals, and Social Relations* [Doctoral dissertation].
- Baker, C. L., Jara-Ettinger, J., Saxe, R., & Tenenbaum, J. B. (2017). Rational quantitative attribution of beliefs, desires and percepts in human mentalizing. *Nature Human Behaviour*, 1(4), 64. <https://doi.org/10.1038/s41562-017-0064>
- Baker, C. L., Saxe, R., & Tenenbaum, J. B. (2009). Action understanding as inverse planning. *Cognition*, 113(3), 329–349. <https://doi.org/10.1016/j.cognition.2009.07.005>
- Baron-Cohen, S., Leslie, A. M., & Frith, U. (1985). Does the autistic child have a “theory of mind” ?. *Cognition*, 21(1), 37–46. [https://doi.org/10.1016/0010-0277\(85\)90022-8](https://doi.org/10.1016/0010-0277(85)90022-8)
- Barone, P., Corradi, G., & Gomila, A. (2019). Infants' performance in spontaneous-response false belief tasks: A review and meta-analysis. *Infant Behavior and Development*, 57, 101350. <https://doi.org/10.1016/j.infbeh.2019.101350>
- Battaglia, P. W., Hamrick, J. B., & Tenenbaum, J. B. (2013). Simulation as an engine of physical scene understanding. *Proceedings of the National Academy of Sciences*, 110(45), 18327–18332. <https://doi.org/10.1073/pnas.1306572110>
- Baum, E. B. (2004). *What is thought?*. MIT Press.
- Beaudoin, C., Leblanc, É., Gagner, C., & Beauchamp, M. H. (2020). Systematic Review and Inventory of Theory of Mind Measures for Young Children. *Frontiers in Psychology*, 10, 2905. <https://doi.org/10.3389/fpsyg.2019.02905>
- Berthiaume, V. G., Shultz, T. R., & Onishi, K. H. (2013). A constructivist connectionist model of transitions on false-belief tasks. *Cognition*, 126(3), 441–458. <https://doi.org/10.1016/j.cognition.2012.11.005>
- Berwick, R. C., Pietroski, P., Yankama, B., & Chomsky, N. (2011). Poverty of the Stimulus Revisited. *Cognitive Science*, 35(7), 1207–1242. <https://doi.org/10.1111/j.1551-6709.2011.01189.x>
- Bowers, M., Olausson, T. X., Wong, L., Grand, G., Tenenbaum, J. B., Ellis, K., & Solar-Lezama, A. (2023). Top-Down Synthesis for Library Learning. *Proceedings of the ACM on Programming Languages*, 7(POPL), 1182–1213. <https://doi.org/10.1145/3571234>
- Bruner, J. S., Goodnow, J. J., & Austin, G. A. (1986). *A study of thinking*. Transaction Books.
- Buttelmann, D., Carpenter, M., & Tomasello, M. (2009). Eighteen-month-old infants show false belief understanding in an active helping paradigm. *Cognition*, 112(2), 337–342. <https://doi.org/10.1016/j.cognition.2009.05.006>
- Butterfill, S. A., & Apperly, I. A. (2013). How to Construct a Minimal Theory of Mind. *Mind & Language*, 28(5), 606–637. <https://doi.org/10.1111/mila.12036>
- Carey, S. (2011). *The Origin of Concepts*. Oxford University Press USA - OSO.

- Chater, N., & Vitányi, P. (2003). Simplicity: a unifying principle in cognitive science?. *Trends in Cognitive Sciences*, 7(1), 19–22. [https://doi.org/10.1016/S1364-6613\(02\)00005-0](https://doi.org/10.1016/S1364-6613(02)00005-0)
- Chomsky, N. (1975). *Reflections on Language*. Pantheon Books. <https://books.google.nl/books?id=R78kAQAAMAAJ>
- Chomsky, N. (2014). *Aspects of the Theory of Syntax* (50th ed). MIT Press.
- Churchland, P. M. (1981). Eliminative Materialism and the Propositional Attitudes. *The Journal of Philosophy*, 78(2), 67. <https://doi.org/10.2307/2025900>
- Coşkun, K., Kara, C., & Coskun, M. (2020). New Scaling of Theory of Mind Tasks: Where Can the Intentional Aspect Be Scaled?. *Activitas Nervosa Superior*, 62(3), 78–87. <https://doi.org/10.1007/s41470-020-00072-2>
- Crane, T. (2015). *The Mechanical Mind* (0th ed.). Routledge. <https://doi.org/10.4324/9781315658681>
- Csibra, G., & Gergely, G. (2007). ‘Obsessed with goals’: Functions and mechanisms of teleological interpretation of actions in humans. *Acta Psychologica*, 124(1), 60–78. <https://doi.org/10.1016/j.actpsy.2006.09.007>
- Dechter, E. (2013). *Bootstrap learning via modular concept discovery* (F. Rossi, Ed.). AAAI Press [u.a.].
- Dennett, D. C. (2002). *The Intentional Stance* (8. pr.). MIT Press.
- Downes, S. M. (1999). Can Scientific Development and Children's Cognitive Development Be the Same Process?. *Philosophy of Science*, 66(4), 565–578. <https://doi.org/10.1086/392755>
- Dörrenberg, S., Rakoczy, H., & Liszkowski, U. (2018). How (not) to measure infant Theory of Mind: Testing the replicability and validity of four non-verbal measures. *Cognitive Development*, 46, 12–30. <https://doi.org/10.1016/j.cogdev.2018.01.001>
- Ellis, K., Wong, C., Nye, M., Sable-Meyer, M., Cary, L., Morales, L., Hewitt, L., Solar-Lezama, A., & Tenenbaum, J. B. (2020, June). *DreamCoder: Growing generalizable, interpretable knowledge with wake-sleep Bayesian program learning*. arXiv. <https://doi.org/10.48550/arXiv.2006.08381>
- Feldman, J. (2000). Minimization of Boolean complexity in human concept learning. *Nature*, 407(6804), 630–633. <https://doi.org/10.1038/35036586>
- Flener, P., & Schmid, U. (2008). An introduction to inductive programming. *Artificial Intelligence Review*, 29(1), 45–62. <https://doi.org/10.1007/s10462-009-9108-7>
- Fodor, J. (1992). A theory of the child's theory of mind. *Cognition*, 44(3), 283–296. [https://doi.org/10.1016/0010-0277\(92\)90004-2](https://doi.org/10.1016/0010-0277(92)90004-2)
- Fodor, J. A. (1998). *Concepts: Where Cognitive Science Went Wrong* (1st ed.). Oxford University PressOxford. <https://doi.org/10.1093/0198236360.001.0001>
- Fodor, J. A. (2010). *The Language of Thought* (Digit. repr). Harvard Univ. Pr.
- Fodor, J. A., & Pylyshyn, Z. W. (1988). Connectionism and cognitive architecture: A critical analysis. *Cognition*, 28(1–2), 3–71. [https://doi.org/10.1016/0010-0277\(88\)90031-5](https://doi.org/10.1016/0010-0277(88)90031-5)

- Gergely, G., & Csibra, G. (2003). Teleological reasoning in infancy: the naïve theory of rational action. *Trends in Cognitive Sciences*, 7(7), 287–292. [https://doi.org/10.1016/S1364-6613\(03\)00128-1](https://doi.org/10.1016/S1364-6613(03)00128-1)
- Gergely, G., Nádasdy, Z., Csibra, G., & Bíró, S. (1995). Taking the intentional stance at 12 months of age. *Cognition*, 56(2), 165–193. [https://doi.org/10.1016/0010-0277\(95\)00661-H](https://doi.org/10.1016/0010-0277(95)00661-H)
- Goldman, A. I. (2006). *Simulating minds: the philosophy, psychology, and neuroscience of mindreading*. Oxford University press.
- Goodman, N. (1983). *Fact, fiction, and forecast* (4. ed). Harvard Univ. Pr.
- Goodman, N. D., Baker, C. L., Bonawitz, E. B., Mansinghka, V. K., Gopnik, A., Wellman, H., Schulz, L., & Tenenbaum, J. B. (2006). Intuitive Theories of Mind: A Rational Approach to False Belief. *Proceedings of the Annual Meeting of the Cognitive Science Society*, 28.
- Goodman, N. D., Tenenbaum, J. B., & Gerstenberg, T. (2014). *Concepts in a Probabilistic Language of Thought*.
- Goodman, N. D., Tenenbaum, J. B., Feldman, J., & Griffiths, T. L. (2008). A Rational Analysis of Rule-Based Concept Learning. *Cognitive Science*, 32(1), 108–154. <https://doi.org/10.1080/03640210701802071>
- Goodman, N. D., Ullman, T. D., & Tenenbaum, J. B. (2011). Learning a theory of causality. *Psychological Review*, 118(1), 110–119. <https://doi.org/10.1037/a0021336>
- Gopnik, A. (1997). *Words, thoughts, and theories*. MIT Press.
- Gopnik, A. (2012). Scientific Thinking in Young Children: Theoretical Advances, Empirical Research, and Policy Implications. *Science*, 337(6102), 1623–1627. <https://doi.org/10.1126/science.1223416>
- Gopnik, A., & Tenenbaum, J. B. (2007). Bayesian networks, Bayesian learning and cognitive development. *Developmental Science*, 10(3), 281–287. <https://doi.org/10.1111/j.1467-7687.2007.00584.x>
- Gopnik, A., & Wellman, H. M. (1992). Why the Child's Theory of Mind Really Is a Theory. *Mind & Language*, 7(1–2), 145–171. <https://doi.org/10.1111/j.1468-0017.1992.tb00202.x>
- Gopnik, A., & Wellman, H. M. (2012). Reconstructing constructivism: Causal models, Bayesian learning mechanisms, and the theory theory. *Psychological Bulletin*, 138(6), 1085–1108. <https://doi.org/10.1037/a0028044>
- Gordon, R. M. (1986). Folk Psychology as Simulation. *Mind & Language*, 1(2), 158–171. <https://doi.org/10.1111/j.1468-0017.1986.tb00324.x>
- Griffiths, T. L., Chater, N., & Tenenbaum, J. B. (2024). *Bayesian models of cognition: reverse engineering the mind*. The MIT Press.
- Griffiths, T. L., Chater, N., Kemp, C., Perfors, A., & Tenenbaum, J. B. (2010). Probabilistic models of cognition: exploring representations and inductive biases. *Trends in Cognitive Sciences*, 14(8), 357–364. <https://doi.org/10.1016/j.tics.2010.05.004>

- Griffiths, T. L., Sobel, D. M., Tenenbaum, J. B., & Gopnik, A. (2011). Bayes and Blickets: Effects of Knowledge on Causal Induction in Children and Adults: Cognitive Science. *Cognitive Science*, 35(8), 1407–1455. <https://doi.org/10.1111/j.1551-6709.2011.01203.x>
- Gulwani, S., Polozov, O., & Singh, R. (2017). *Program synthesis* (Issue 4.2017, 1-2). Now Publishers.
- Heal, J. (2004). Replication and Functionalism. In *Philosophy of mind: a guide and anthology: Philosophy of mind: a guide and anthology* (1. publ, 7. [Nachdr.]). Oxford Univ. Press.
- Heider, F., & Simmel, M. (1944). An Experimental Study of Apparent Behavior. *The American Journal of Psychology*, 57(2), 243. <https://doi.org/10.2307/1416950>
- Heyes, C. M. (2018). *Cognitive gadgets: the cultural evolution of thinking*. Harvard University press.
- Heyes, C. M., & Frith, C. D. (2014). The cultural evolution of mind reading. *Science*, 344(6190), 1243091. <https://doi.org/10.1126/science.1243091>
- Jara-Ettinger, J., & Rubio-Fernandez, P. (2021). Quantitative mental state attributions in language understanding. *Science Advances*, 7(47), eabj970. <https://doi.org/10.1126/sciadv.abj0970>
- Jaynes, E. T., & Bretthorst, G. L. (2003). *Probability theory: the logic of science*. Cambridge university press.
- Kemp, C., Tenenbaum, J. B., Niyogi, S., & Griffiths, T. L. (2010). A probabilistic model of theory formation. *Cognition*, 114(2), 165–196. <https://doi.org/10.1016/j.cognition.2009.09.003>
- Kulke, L., Von Duhn, B., Schneider, D., & Rakoczy, H. (2018). Is Implicit Theory of Mind a Real and Robust Phenomenon? Results From a Systematic Replication Study. *Psychological Science*, 29(6), 888–900. <https://doi.org/10.1177/0956797617747090>
- Lake, B. M., Ullman, T. D., Tenenbaum, J. B., & Gershman, S. J. (2016, November). *Building Machines That Learn and Think Like People*. arXiv. <https://doi.org/10.48550/arXiv.1604.00289>
- Leslie, A. M. (1987). Pretense and representation: The origins of "theory of mind.". *Psychological Review*, 94(4), 412–426. <https://doi.org/10.1037/0033-295X.94.4.412>
- Leslie, A. M. (1994). Pretending and believing: issues in the theory of ToMM. *Cognition*, 50(1–3), 211–238. [https://doi.org/10.1016/0010-0277\(94\)90029-9](https://doi.org/10.1016/0010-0277(94)90029-9)
- Leslie, A. M. (2005). Developmental parallels in understanding minds and bodies. *Trends in Cognitive Sciences*, 9(10), 459–462. <https://doi.org/10.1016/j.tics.2005.08.002>
- Liang, P., Jordan, M. I., & Klein, D. (2010). *Learning Programs: A Hierarchical Bayesian Approach*.
- MacKay, D. J. C. (2019). *Information theory, inference, and learning algorithms* (22nd printing). Cambridge University Press.
- Margolis, E., Samuels, R., & Stich, S. P. (Eds.). (2012). *The Oxford handbook of philosophy of cognitive science*. Oxford University Press. <https://doi.org/10.1093/oxfordhb/9780195309799.001.0001>
- Marr, D. (2010). *Vision: A Computational Investigation into the Human Representation and Processing of Visual Information*. The MIT Press. <https://doi.org/10.7551/mitpress/9780262514620.001.0001>

- McClelland, J. L. (2010). Emergence in Cognitive Science. *Topics in Cognitive Science*, 2(4), 751–770. <https://doi.org/10.1111/j.1756-8765.2010.01116.x>
- Newcombe, N. S. (2011). What Is Neoconstructivism?. *Child Development Perspectives*, 5(3), 157–160. <https://doi.org/10.1111/j.1750-8606.2011.00180.x>
- Newell, A., Shaw, J. C., & Simon, H. A. (1958). Elements of a theory of human problem solving. *Psychological Review*, 65(3), 151–166. <https://doi.org/10.1037/h0048495>
- Nichols, S., & Stich, S. P. (2003). *Mindreading: an integrated account of pretence, self-awareness, and understanding other minds*. Clarendon. <https://doi.org/10.1093/0198236107.001.0001>
- Onishi, K. H., & Baillargeon, R. (2005). Do 15-Month-Old Infants Understand False Beliefs?. *Science*, 308(5719), 255–258. <https://doi.org/10.1126/science.1107621>
- Palmarini, A. B., Lucas, C. G., & Siddharth, N. (2024, May). *Bayesian Program Learning by Decompling Amortized Knowledge*. arXiv. <https://doi.org/10.48550/arXiv.2306.07856>
- Perfors, A., Tenenbaum, J. B., & Wonnacott, E. (2010). Variability, negative evidence, and the acquisition of verb argument constructions. *Journal of Child Language*, 37(3), 607–642. <https://doi.org/10.1017/S0305000910000012>
- Perner, J. (1991). *Understanding the Representational Mind*. MIT Press.
- Perner, J., & Wimmer, H. (1985). “John thinks that Mary thinks that...” attribution of second-order beliefs by 5- to 10-year-old children. *Journal of Experimental Child Psychology*, 39(3), 437–471. [https://doi.org/10.1016/0022-0965\(85\)90051-7](https://doi.org/10.1016/0022-0965(85)90051-7)
- Piantadosi, S. T., Goodman, N. D., Ellis, B. A., & Tenenbaum, J. B. (n.d.). *A Bayesian Model of the Acquisition of Compositional Semantics*.
- Piantadosi, S. T., Tenenbaum, J. B., & Goodman, N. D. (2012). Bootstrapping in a language of thought: A formal model of numerical concept learning. *Cognition*, 123(2), 199–217. <https://doi.org/10.1016/j.cognition.2011.11.005>
- Polivy, D. (n.d.). *EMPIRICISM AND THE PHILOSOPHY OF MIND*.
- Quine, W. V. (2013). *Word and object* (New ed). MIT Press.
- Rakoczy, H. (2022). Foundations of theory of mind and its development in early childhood. *Nature Reviews Psychology*, 1(4), 223–235. <https://doi.org/10.1038/s44159-022-00037-z>
- Ruffman, T. (2014). To belief or not belief: Children’s theory of mind. *Developmental Review*, 34(3), 265–293. <https://doi.org/10.1016/j.dr.2014.04.001>
- Rule, J. S. (n.d.). *Pre ’cis of The child as hacker: Building more human-like models of learning*.
- Rumelhart, D. E. (1986). *Parallel Distributed Processing: Explorations in the Microstructure of Cognition: Foundations* (Issue v.1). MIT Press.
- Schmid, U., Kitzelmann, E., & Plasmeijer, R. (n.d.). *AAIP 2009 Proceedings of the ACM SIGPLAN Workshop*.
- Schmidt, L. A. (n.d.). *Meaning and compositionality as statistical induction of categories and constraints*.

- Scholl, B. J., & Leslie, A. M. (2001). Minds, Modules, and Meta-Analysis. *Child Development*, 72(3), 696–701. <https://doi.org/10.1111/1467-8624.00308>
- Schulz, L. (2012). The origins of inquiry: inductive inference and exploration in early childhood. *Trends in Cognitive Sciences*, 16(7), 382–389. <https://doi.org/10.1016/j.tics.2012.06.004>
- Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal*, 27(3), 379–423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>
- Shum, M., Kleiman-Weiner, M., Littman, M. L., & Tenenbaum, J. B. (2019). Theory of Minds: Understanding Behavior in Groups through Inverse Planning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 33(1), 6163–6170. <https://doi.org/10.1609/aaai.v33i01.33016163>
- Siskind, J. M. (n.d.). *A computational study of cross-situational techniques for learning word-to-meaning mappings*.
- Southgate, V., Senju, A., & Csibra, G. (2007). Action Anticipation Through Attribution of False Belief by 2-Year-Olds. *Psychological Science*, 18(7), 587–592. <https://doi.org/10.1111/j.1467-9280.2007.01944.x>
- Spelke, E. S. (2022). *What babies know: core knowledge and composition, volume 1* (First Edition). Oxford University Press. <https://doi.org/10.1093/oso/9780190618247.001.0001>
- Spelke, E. S., & Kinzler, K. D. (2007). Core knowledge. *Developmental Science*, 10(1), 89–96. <https://doi.org/10.1111/j.1467-7687.2007.00569.x>
- Tenenbaum, J. B., Kemp, C., Griffiths, T. L., & Goodman, N. D. (2011). How to Grow a Mind: Statistics, Structure, and Abstraction. *Science*, 331(6022), 1279–1285. <https://doi.org/10.1126/science.1192788>
- Ullman, T. D., & Tenenbaum, J. B. (n.d.). *Bayesian Models of Conceptual Development: Learning as Building Models of the World*.
- Ullman, T. D., Goodman, N. D., & Tenenbaum, J. B. (2010). Theory Acquisition as Stochastic Search. *Proceedings of the Thirty-Second Annual Conference of the Cognitive Science Society*.
- Wellman, H. M. (2018). Theory of mind: The state of the art. *European Journal of Developmental Psychology*, 15(6), 728–755. <https://doi.org/10.1080/17405629.2018.1435413>
- Wellman, H. M., & Gelman, S. A. (1992). Cognitive Development: Foundational Theories of Core Domains. In *Annual review of psychology. Volume 43, 1992: Annual review of psychology. Volume 43, 1992*. Annual Reviews Inc.
- Wellman, H. M., & Liu, D. (2004). Scaling of Theory-of-Mind Tasks. *Child Development*, 75(2), 523–541. <https://doi.org/10.1111/j.1467-8624.2004.00691.x>
- Wellman, H. M., Carey, S., Gleitman, L., Newport, E. L., & Spelke, E. S. (1990). *The Child's Theory of Mind*. The MIT Press. <https://doi.org/10.7551/mitpress/1811.001.0001>
- Wellman, H. M., Cross, D., & Watson, J. (2001). Meta-Analysis of Theory-of-Mind Development: The Truth about False Belief. *Child Development*, 72(3), 655–684. <https://doi.org/10.1111/1467-8624.00304>

- Wellman, H. M., Kushnir, T., Xu, F., & Brink, K. A. (2016). Infants Use Statistical Sampling to Understand the Psychological World. *Infancy*, 21(5), 668–676. <https://doi.org/10.1111/inf.12131>
- Wimmer, H., & Perner, J. (1983). Beliefs about beliefs: Representation and constraining function of wrong beliefs in young children's understanding of deception. *Cognition*, 13(1), 103–128. [https://doi.org/10.1016/0010-0277\(83\)90004-5](https://doi.org/10.1016/0010-0277(83)90004-5)
- Woodward, A. (1998). Infants selectively encode the goal object of an actor's reach. *Cognition*, 69(1), 1–34. [https://doi.org/10.1016/S0010-0277\(98\)00058-4](https://doi.org/10.1016/S0010-0277(98)00058-4)
- Xu, F. (2019). Towards a rational constructivist theory of cognitive development. *Psychological Review*, 126(6), 841–864. <https://doi.org/10.1037/rev0000153>
- Xu, F., & Kushnir, T. (2013). Infants Are Rational Constructivist Learners. *Current Directions in Psychological Science*, 22(1), 28–32. <https://doi.org/10.1177/0963721412469396>
- Xu, F., Kushnir, T., & Benson, J. B. (2012). *Rational constructivism in cognitive development* (Issue 43). Academic Press.
- Zettlemoyer, L. S., & Collins, M. (n.d.). *Learning to Map Sentences to Logical Form: Structured Classification with Probabilistic Categorical Grammars*.